

Chapter 2

1. Deterministic Autoencoder (AE)

Flow:

$$x \rightarrow z \rightarrow \hat{x}$$

- Encoder outputs one latent vector.
- Decoder reconstructs the input.
- AE minimizes reconstruction error only.

Advantages:

- Compression
- Denoising
- Dimensionality reduction

Problems:

- No probabilistic latent space
- Latent space contains holes
- Random sampling produces unrealistic outputs
- Not a true generative model

AE learns reconstruction, but not the probability distribution of data.

2. Variational Autoencoder (VAE)

Flow:

$$x \rightarrow (\mu, \sigma^2) \rightarrow z \rightarrow \hat{x}$$

Encoder outputs:

- Mean μ
- Variance σ^2

Latent sampling:

$$z \sim N(\mu, \sigma^2)$$

Main idea:

VAE learns a probability distribution instead of one deterministic latent vector.

Advantages:

- Smooth latent space

- Meaningful sampling
- Can generate realistic new data
- Probabilistic generative model

3. Why VAE is Needed

AE reconstructs well but fails in generation.

Reason:

- Latent vectors occupy arbitrary disconnected regions.
- Random samples from latent space may fall into holes.
- Decoder was never trained in those regions.

VAE solves this by forcing latent variables to follow:

$$p(z) = \mathcal{N}(0, I)$$

Effects:

- Smooth latent space
- Nearby points generate similar outputs
- Random sampling becomes meaningful

4. Generative Process

Step 1:

Sample latent vector:

$$z \sim p(z) = \mathcal{N}(0, I)$$

Step 2:

Generate data:

$$x \sim p_{\theta}(x|z)$$

Definitions:

$p(z)$: prior distribution

$p_{\theta}(x|z)$: decoder likelihood

$q\phi(z|x)$: encoder approximation to posterior

5. Marginal Likelihood

Goal:

$$p(x) = \int p(x|z)p(z)dz$$

Training objective:
maximize $\log p(x)$

Problem:

- Integral is intractable
- Decoder is nonlinear
- High-dimensional latent space

Solution:

Optimize ELBO instead of direct likelihood.

6. ELBO (Evidence Lower Bound)

Formula:

ELBO =

$$\mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x) || p(z))$$

ELBO is a lower bound on:

$$\log p(x)$$

Reason:

$$\log p(x) \geq \text{ELBO}$$

The closer ELBO is to $\log p(x)$, the better the model.

7. Reconstruction Term

$$\mathbb{E}_{q(z|x)}[\log p(x|z)]$$

Measures:

- reconstruction quality

Interpretation:

“How well can decoder reconstruct x from latent variables z ?”

Large value:

- good reconstruction

8. KL Divergence Term

$$\text{KL}(q(z|x) || p(z))$$

Measures:

difference between:

- encoder distribution
- prior distribution

Purpose:

- regularize latent space
- force latent variables toward $N(0, I)$
- improve sampling quality

Without KL regularization:

VAE behaves similarly to deterministic AE.

9. Tension in VAE

There is a tradeoff between:

- reconstruction quality
- latent space regularization

Too little KL:

- messy latent space
- poor generation

Too much KL:

- latent bottleneck too restrictive
- blurry reconstruction

10. Jensen's Inequality

Used to derive ELBO.

Since log is concave:

$$\log(E[x]) \geq E[\log(x)]$$

Transforms:

hard log of expectation

into

easier expectation of log

Therefore:

$$\log p(x) \geq \text{ELBO}$$

11. ELBO Gap

Relationship:

$$\log p(x) - \text{ELBO} = \text{KL}(q(z|x) || p(z|x))$$

If:

$$q(z|x) = p(z|x)$$

Then:

$$\text{ELBO} = \log p(x)$$

Meaning:

ELBO becomes exact when approximate posterior equals true posterior.

12. Reparameterization Trick

Problem:

Sampling is random:

$$z \sim N(\mu, \sigma^2)$$

Random sampling blocks gradient flow.

Solution:

$$z = \mu + \sigma \varepsilon$$

where:

$$\varepsilon \sim N(0, I)$$

Advantages:

- separates randomness from parameters
- makes sampling differentiable
- allows backpropagation
- enables stable VAE training

Conceptually:

Transforms random sampling into differentiable computation.

13. Encoder Equations

Encoder outputs:

$$\mu\phi(x), \log\sigma^2\phi(x)$$

Variational posterior:

$$q\phi(z|x)=N(\mu\phi(x),\text{diag}(\sigma^2\phi(x)))$$

Encoder learns:

- latent mean
- latent uncertainty

14. Decoder Equations

Decoder defines:

$$p\theta(x|z)$$

Bernoulli decoder:

- binary data
- BCE loss

Gaussian decoder:

- real-valued data
- MSE loss

15. VAE Loss Function

Loss =

Reconstruction Loss + KL

Equivalent to:

$$\text{Loss} = -\text{ELBO}$$

Reconstruction:

improves fidelity

KL:

regularizes latent space

16. β -VAE

β -VAE modifies ELBO:

$$L_{\beta\text{-VAE}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta \text{KL}(q(z|x) || p(z))$$

If $\beta > 1$:

- stronger regularization
- encourages disentanglement
- more compressed latent space

Disadvantage:

- reconstruction quality may degrade

17. Disentanglement

Disentanglement means:

different latent variables learn separate semantic factors.

Example:

- one variable controls rotation
- another controls thickness
- another controls style

Standard VAE does not guarantee disentanglement automatically.

18. Conditional VAE (CVAE)

CVAE conditions encoder and decoder on label y .

Encoder:

$$q(z|x,y)$$

Decoder:

$$p(x|z,y)$$

Advantages:

- controllable generation
- reduced entanglement

- better structured latent space

19. Posterior Collapse

Posterior collapse occurs when:

$$q(z|x) \approx p(z)$$

Meaning:

- latent variables carry little information
- decoder ignores latent space

Causes:

- decoder too powerful
- KL pressure too strong

Effects:

- weak latent representation
- poor reconstruction details

20. Key Differences Between AE and VAE

AE:

- deterministic
- one latent vector
- reconstruction only
- bad sampling

VAE:

- probabilistic
- latent distribution
- reconstruction + generation
- smooth Gaussian latent space
- meaningful sampling

21. Most Important Equations

Sampling:

$$z = \mu + \sigma \epsilon$$

$$\epsilon \sim N(0, I)$$

ELBO:

ELBO =
 $E_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x)||p(z))$

ELBO Gap:
 $\log p(x) = \text{ELBO} + \text{KL}(q(z|x)||p(z|x))$

Marginal Likelihood:
 $p(x) = \int p(x|z)p(z)dz$

22. Main Idea of VAE

VAE combines:

- deep learning
- probabilistic inference
- latent-variable generative modeling

Goal:

learn a latent space that supports:

- reconstruction
- generation
- interpolation
- sampling
- controllable representation learning

Sample Questions and Answers

Question 1

Why is the ELBO called a lower bound on $\log p(x)$, and under what condition does it become exact?

Sample Answer:

The ELBO is called a lower bound because:

$$\log p(x) = \text{ELBO} + \text{KL}(q(z|x) || p(z|x))$$

Since KL divergence is always nonnegative:

$$\text{ELBO} \leq \log p(x)$$

The bound becomes exact when:

$$q(z|x) = p(z|x)$$

In that case the KL gap becomes zero.

Question 2

Interpret the two main terms in the ELBO and explain the tension between them.

Sample Answer:

The reconstruction term encourages the model to reconstruct the input accurately using latent variables.

The KL term regularizes the latent space and pushes the approximate posterior toward the prior distribution.

The tension is that:

- better reconstruction stores more information in z
- stronger KL pressure compresses latent representation

The VAE must balance reconstruction fidelity and latent space regularization.

Question 3

Explain the reparameterization trick conceptually.

Sample Answer:

The reparameterization trick rewrites latent sampling as:

$$z = \mu + \sigma \epsilon$$

where:

$$\epsilon \sim N(0, I)$$

This separates randomness from learnable parameters.

Now z becomes a differentiable function, allowing gradients to flow through the latent sample during backpropagation.

This is one of the key ideas that makes VAE training practical.

Question 4

Why does standard VAE not automatically guarantee disentanglement?

Sample Answer:

Standard VAE only optimizes reconstruction and regularization.

These objectives alone do not force latent variables to represent separate semantic factors.

β -VAE encourages disentanglement by increasing KL pressure.

CVAE improves structure by conditioning on known information such as labels.

Question 5

What is posterior collapse?

Sample Answer:

Posterior collapse occurs when:

$$q(z|x) \approx p(z)$$

This means latent variables carry little information about the input.

The decoder ignores z and reconstructs mostly using learned generic patterns.

Causes include:

- very strong KL pressure
- overly powerful decoder

Consequences:

- weak latent representation
- poor informative encoding

Chapter 3

1. Core Idea of Diffusion Models

Diffusion models are generative models that learn to generate data by reversing a gradual noising process.

Instead of generating an image in one step:

- start from Gaussian noise
- iteratively denoise

Main intuition:

Learning many small denoising steps is easier than learning one direct mapping from noise to realistic data.

Trajectory:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

where:

- x_0 = clean data
- $x_T \approx$ Gaussian noise

2. Diffusion vs VAE

VAE:

- uses low-dimensional latent variable z
- decodes from latent representation

Diffusion:

- latent structure is an entire trajectory
- gradually transforms data into noise and back

Diffusion generation is iterative instead of one-shot decoding.

3. Markov Chains

A diffusion process is a Markov chain.

Markov property:

Future depends only on present state.

Mathematically:

$$p(x_t | x_{t-1}, \dots, x_0) = p(x_t | x_{t-1})$$

Forward chain:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

Reverse chain:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$$

4. Forward Diffusion Process

Forward process gradually destroys information.

Transition:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I)$$

Equivalent sampling equation:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{(1 - \alpha_t)} \varepsilon$$

where:

$$\varepsilon \sim N(0, I)$$

Each step:

- shrinks signal
- adds Gaussian noise

After many steps:

$$x_T \approx N(0, I)$$

5. Why Forward Process Matters

Forward process:

- defines controlled path from data to noise
- is fixed and analytic
- does not require learning

Main purpose:

create a tractable corruption process that can later be reversed.

6. Trajectory vs Marginal

Trajectory distribution:

$$q(x_{1:T} | x_0)$$

Represents:

entire noisy path.

Marginal distribution:

$$q(x_t | x_0)$$

Represents:

state at one timestep only.

Training usually uses marginals instead of simulating the full trajectory.

Closed-form marginal:

$$q(x_t | x_0) = N(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

where:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

7. Why Closed-Form Marginal is Important

Closed-form marginal allows:

- direct sampling at arbitrary timestep
- efficient training
- no need to simulate all intermediate steps

Training pair:

$$(x_0, x_t)$$

can be generated directly.

8. Reverse Process

Goal:

reverse the noising process.

Ideal reverse conditional:

$$q(x_{t-1} | x_t)$$

Problem:
depends on unknown data distribution.

Therefore:
cannot compute exactly for real data.

Solution:
learn approximation using neural network.

9. Learned Reverse Process

Diffusion defines learned reverse chain:

$$p_{\theta}(x_{0:T}) = p(x_T) \prod p_{\theta}(x_{t-1} | x_t)$$

Usually:
 $p_{\theta}(x_{t-1} | x_t) = N(x_{t-1} ; \mu_{\theta}(x_t, t), \Sigma_t)$

Neural network predicts:

- reverse mean
- or noise
- or x_0

Most common:
predict added noise.

10. Noise Prediction Objective

Forward marginal:
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

Neural network predicts:
 $\varepsilon_{\theta}(x_t, t) \approx \varepsilon$

Loss:
$$L_{\text{simple}} = E || \varepsilon - \varepsilon_{\theta}(x_t, t) ||^2$$

Why predicting noise works:

- mathematically related to score estimation
- estimates denoising direction

- stable optimization

11. Score-Based Interpretation

Diffusion models are also score-based models.

Score:

$$\nabla_x \log p(x)$$

Meaning:

direction toward regions of higher data density.

Predicting noise is closely related to estimating the score function.

12. Why Timestep Conditioning is Needed

Denoising difficulty changes with timestep.

Small t:

- slightly noisy
- semantic structure preserved

Large t:

- almost pure noise
- difficult denoising

Without timestep information:

denoising becomes ambiguous.

Therefore network must know timestep t.

13. Sinusoidal Time Embeddings

Instead of feeding integer t directly:
convert t into vector embedding.

Uses sinusoidal functions:

similar to Transformer positional encoding.

Advantages:

- nearby timesteps get nearby embeddings

- expressive across large time range
- helps neural network understand noise level

14. Why U-Net is Used

U-Net is ideal for image diffusion because it combines:

- global structure
- local detail
- multiscale information

Components:

- downsampling path
- upsampling path
- skip connections

Skip connections preserve spatial information during denoising.

15. Noise Schedule

Noise schedule controls:

- how fast information is destroyed
- signal-to-noise ratio (SNR)
- training stability
- sample quality

Definitions:

α_t = remaining signal

β_t = added noise variance

Relation:

$$\alpha_t = 1 - \beta_t$$

Cumulative signal:

$$\bar{\alpha}_t = \prod \alpha_s$$

16. Signal-to-Noise Ratio (SNR)

SNR:

$$\text{SNR}_t = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$$

Large SNR:

- x_t close to x_0
- easy denoising

Small SNR:

- x_t close to pure noise
- difficult denoising

17. Linear vs Cosine Schedule

Linear schedule:

β_t increases linearly.

Simple but not always optimal.

Cosine schedule:

uses smoother signal decay.

Advantages:

- better information allocation
- improved sample quality
- smoother SNR decay

18. Why Reverse Process is Hard

True reverse conditional:

$$q(x_{t-1} | x_t)$$

depends on:

- $q(x_t)$
- $q(x_{t-1})$
- unknown data distribution

Even though forward transition is Gaussian,
reverse conditional becomes intractable for real data.

Therefore:

learn approximation using neural network.

19. DDPM vs DDIM

DDPM:

- stochastic reverse process
- many sampling steps
- high diversity

DDIM:

- more deterministic sampling
- fewer steps
- faster generation

Important concept:

training and sampling are separate ideas.

20. Families of Diffusion Models

Main diffusion families:

1. DDPM

Classic diffusion model.

2. DDIM

Fast deterministic sampling.

3. Score-SDE

Continuous-time score-based diffusion.

4. Latent Diffusion

Diffusion in latent space.

Used in Stable Diffusion.

5. Guided Diffusion

Uses guidance during sampling.

6. Rectified Flow / Flow Matching

ODE-based deterministic generation.

21. Posterior and Reverse Approximation

True posterior:

$$q(x_{t-1} | x_t, x_0)$$

is tractable during training because x_0 is known.

At generation time:

x_0 unknown.

Therefore:

must learn reverse approximation:

$$p_{\theta}(x_{t-1} | x_t)$$

22. DDPM Training Procedure

Training loop:

1. Sample x_0
2. Sample timestep t
3. Sample Gaussian noise ϵ
4. Create noisy x_t
5. Predict ϵ using neural network
6. Minimize MSE loss

Training data:

$$(x_t, t, \epsilon)$$

23. Information Decay

Forward diffusion destroys information gradually.

Mutual information:

$$I(X_0 ; X_t)$$

decreases with timestep t .

Interpretation:

x_t gradually forgets x_0 as noise increases.

24. Geometric Interpretation

Forward diffusion:

- contracts signal
- increases entropy
- smooths data distribution

Reverse diffusion:

moves noisy samples toward regions of high data density.

Generation:

progressive movement from noise toward data manifold.

25. Why Diffusion Models Work

Diffusion works because:

- forward process is simple and tractable
- reverse process is broken into small local denoising steps
- neural network approximates reverse dynamics

Generation becomes:

sequence of manageable denoising operations instead of one difficult mapping.

26. Most Important Equations

Forward transition:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I)$$

Sampling equation:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{(1 - \alpha_t)} \epsilon$$

Closed-form marginal:

$$q(x_t | x_0) = N(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

Noise prediction loss:

$$L_{\text{simple}} = E || \epsilon - \epsilon_{\theta}(x_t, t) ||^2$$

Signal-to-noise ratio:

$$\text{SNR}_t = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$$

Chapter 3 Sample Questions and Answers

Question 1

What is the conceptual difference between $q(x_{1:T} | x_0)$ and $q(x_t | x_0)$, and why is the distinction important?

Sample Answer:

$q(x_{1:T} | x_0)$ represents the probability distribution over the entire noisy trajectory starting from x_0 .

$q(x_t | x_0)$ represents only the noisy state at a single timestep t .

This distinction matters because training usually needs only the marginal $q(x_t | x_0)$, which has a closed-form Gaussian expression. This allows efficient direct sampling without simulating the full trajectory.

Question 2

Why is the true reverse conditional not directly usable?

Sample Answer:

The reverse conditional depends on the unknown data distribution.

Even though the forward process is Gaussian and tractable, Bayes' rule introduces complicated marginal distributions involving real data.

Therefore, the exact reverse conditional becomes intractable and must be approximated using a neural network.

Question 3

Why does predicting added noise make sense?

Sample Answer:

The noisy sample can be written as:

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{(1-\alpha_t)} \epsilon$$

The unknown component is the noise ϵ .

Predicting ϵ tells the model how far x_t moved away from the clean data structure.

This is also mathematically related to score estimation and denoising direction learning.

Question 4

Why must the denoising network know timestep t ?

Sample Answer:

Small timesteps contain mostly clean signal, while large timesteps contain mostly noise.

The denoising task changes dramatically across timesteps.

Without timestep conditioning, the denoising mapping becomes ambiguous.

Time embeddings tell the network the current signal-to-noise level.

Question 5

Why are diffusion models described as both latent-variable models and score-based models?

Sample Answer:

Diffusion models are latent-variable models because the entire trajectory acts as latent variables connecting noise to data.

They are also score-based models because the denoiser implicitly estimates score-like quantities that guide samples toward high-density data regions.

The two interpretations describe different views of the same process.

Chapter 4 - Advanced Diffusion Concepts

1. Big Picture of Chapter 4

Chapter 4 extends diffusion models from simple denoising systems into a full probabilistic and geometric framework.

Main ideas:

- diffusion training is connected to ELBO optimization
- diffusion models learn score fields
- reverse diffusion is drift + stochasticity
- guided diffusion allows controllable generation
- classifier guidance and classifier-free guidance modify the score field
- FID evaluates generation quality in feature space

2. ELBO and Diffusion Training

Diffusion models maximize data likelihood indirectly through the Evidence Lower Bound (ELBO).

Main likelihood:

$$\log p_{\theta}(x_0)$$

Since exact likelihood is intractable:
optimize ELBO instead.

ELBO decomposes into:

- reconstruction term
- KL divergence terms
- prior matching term

Important idea:

once Gaussian assumptions are used, the KL terms reduce to mean matching.

3. Why MSE Loss Appears

The famous diffusion MSE loss is not arbitrary.

Forward marginal:

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{(1-\alpha_t)} \epsilon$$

The model predicts:

$$\epsilon_{\theta}(x_t, t)$$

Training loss:

$$L_{\text{simple}} = E || \epsilon - \epsilon_{\theta}(x_t, t) ||^2$$

This loss emerges naturally from ELBO optimization after reparameterizing the reverse mean using noise prediction.

Therefore:

noise prediction = variational inference.

4. What is a Score Function?

Score:

$$s(x) = \nabla_x \log p(x)$$

Meaning:

direction of steepest increase in probability density.

The score tells us:

- where probability increases
- how strongly probability increases

For a Gaussian:

score points toward the mean.

For multimodal distributions:

score points toward the locally most plausible mode.

5. Gaussian Score Example

For:

$$p(x) = N(\mu, \sigma^2)$$

Log-density:

$$\log p(x) = C - (x - \mu)^2 / (2\sigma^2)$$

Score:

$$s(x) = -(x - \mu) / \sigma^2$$

Interpretation:

- if $x > \mu \rightarrow$ score negative $\rightarrow$ move left
- if $x < \mu \rightarrow$ score positive $\rightarrow$ move right

Therefore:

score always pulls toward the mean.

6. Bimodal Distribution Score

For bimodal distributions:

the score becomes a weighted combination of the scores of both modes.

Near one mode:

that mode dominates the score.

Between modes:

score becomes a compromise.

Important intuition:

the score field points toward the nearest plausible peak.

7. Denoising-Score Identity

A major theoretical result:

$$\nabla_{x_t} \log p_t(x_t) \approx -\epsilon_{\theta}(x_t, t) / \sqrt{(1-\bar{\alpha}_t)}$$

Meaning:

noise prediction is equivalent to learning the score field.

Therefore:

even though DDPM predicts noise,
it is implicitly learning probability geometry.

8. Reverse Diffusion as Drift + Noise

Reverse diffusion update:

$$x_{t-1} = x_t + \beta_t s_t(x_t) + \sqrt{\beta_t} z$$

where:

- $\beta_t \text{st}(\mathbf{x}_t)$ = deterministic drift
- $\sqrt{\beta_t} \mathbf{z}$ = stochastic diffusion noise

Interpretation:

- drift term denoises
- stochastic term preserves diversity

Without noise:

sampling collapses to mode-seeking optimization.

9. Why Reverse Diffusion Needs Noise

If only the score step is used:

samples would follow gradient ascent and collapse to peaks.

Adding Gaussian noise:

- preserves sampling diversity
- maintains distribution coverage
- prevents mode collapse

Diffusion is therefore:

controlled stochastic transport.

10. Guided Diffusion

Unconditional diffusion learns:

$p(\mathbf{x})$

But most applications need:

$p(\mathbf{x}|\mathbf{y})$

Examples:

- class-conditional generation
- text-to-image
- style transfer
- semantic control

Guided diffusion modifies the score field so samples move toward condition-consistent regions.

11. Geometric Interpretation of Guidance

Unconditional score:
points toward plausible data.

Guided score:
points toward plausible data satisfying condition y .

Guidance changes the vector field followed during reverse diffusion.

12. Classifier Guidance

Classifier guidance uses Bayes' rule:

$$\nabla_x \log p(x|y) = \nabla_x \log p(x) + \nabla_x \log p(y|x)$$

Guided score:
 $s_{\text{guided}} = s_t(x) + w \nabla_x \log p(y|x, t)$

Interpretation:

- diffusion score ensures realism
- classifier gradient enforces condition alignment
- w controls guidance strength

13. Why Noise-Aware Classifiers Are Needed

During reverse diffusion:
classifier sees noisy states x_t instead of clean images x_0 .

A clean-image classifier may fail at large noise levels.

Therefore:
train classifier using noisy samples generated from the same forward diffusion process.

Noise-aware classifier learns:
 $p(y|x_t, t)$

This gives stable and meaningful gradients.

14. Classifier-Free Guidance (CFG)

CFG avoids external classifiers.

Single diffusion model learns:

- conditional prediction
- unconditional prediction

Training trick:

randomly drop conditions during training.

Sampling formula:

$$\epsilon_{\text{cfg}} = (1+w)\epsilon_{\theta}(x_t, t, y) - w\epsilon_{\theta}(x_t, t, \emptyset)$$

Interpretation:

amplify the difference between conditional and unconditional predictions.

15. Why Random Condition Dropping is Important

Without dropping conditions:

model would never learn unconditional behavior.

Random dropping forces the network to learn:

- conditional branch
- unconditional branch

CFG depends on subtracting these two behaviors.

16. Tradeoff in Guidance Strength

Increasing guidance strength:

- improves condition adherence
- reduces diversity
- may create artifacts

Therefore:

guidance is a tradeoff between:

- realism
- controllability
- diversity

17. Drift + Guidance Interpretation

Guided reverse diffusion combines:

- unconditional realism
- conditional steering
- stochastic exploration

The sampler follows:

high-probability AND condition-consistent regions.

18. Time Injection and FiLM Bias

Diffusion models must know timestep t .

Different timesteps:

- contain different noise levels
- require different denoising behavior

Time embedding is injected into feature maps using FiLM-like bias.

Simplified FiLM:

$$h' = h + b(t)$$

This gives:

time-dependent feature processing.

19. Why Time Embeddings Matter

Without timestep conditioning:

network cannot distinguish:

- highly noisy images
- nearly clean images

Time embeddings allow:

- timestep-aware denoising
- smooth behavior across noise levels
- adaptive score estimation

20. DDPM Reverse Step

Reverse mean:

$$\mu_{\theta}(x_t, t) = (x_t - \beta_t / \sqrt{1 - \bar{\alpha}_t} * \epsilon_{\theta}(x_t, t)) / \sqrt{\alpha_t}$$

Sampling:

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sqrt{\beta_t} z$$

Important:

reverse diffusion combines:

- denoising drift
- calibrated randomness

21. q_sample Function

Forward diffusion sampling:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

Purpose:

generate noisy samples directly without simulating all steps.

Advantages:

- efficient training
- random timestep training
- direct access to arbitrary noise levels

22. Why Diffusion Models Learn Geometry

Diffusion does not only memorize images.

The model learns:

- structure of probability density
- score directions
- geometry of noisy distributions

Thus:

diffusion learns how probability mass flows through space.

23. FID (Fréchet Inception Distance)

FID compares real and generated samples in feature space.

Formula:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Lower FID:

generated samples closer to real distribution.

FID compares:

- feature means
- feature covariance

24. Limitations of FID

FID is useful but imperfect.

Limitations:

- depends on feature extractor
- assumes Gaussian feature distributions
- sensitive to sample size
- may miss prompt alignment or controllability

Therefore:

FID should not be the only evaluation metric.

25. Difference Between Classifier Guidance and CFG

Classifier Guidance:

- uses external classifier
- requires backpropagation during sampling
- slower
- works with pretrained unconditional model

CFG:

- no external classifier
- uses two diffusion predictions
- faster
- widely used in modern diffusion systems

26. Reverse Diffusion Interpretation

Reverse diffusion is not:

- simple denoising
- pure random sampling

It is:

stochastic transport from noise toward data distributions.

27. Important Hyperparameters

Key hyperparameters:

- guidance strength w
- number of diffusion steps
- noise schedule
- classifier robustness
- condition drop probability

Changing these affects:

- realism
- diversity
- stability
- controllability

28. Most Important Equations

Score:

$$s(x) = \nabla_x \log p(x)$$

Noise prediction loss:

$$L_{\text{simple}} = E || \epsilon - \epsilon_{\theta}(x_t, t) ||^2$$

Denoising-score identity:

$$\nabla_{x_t} \log p_t(x_t) \approx - \epsilon_{\theta}(x_t, t) / \sqrt{(1 - \bar{\alpha}_t)}$$

Reverse update:

$$x_{t-1} = x_t + \beta_t s_t(x_t) + \sqrt{\beta_t} z$$

Classifier guidance:

$$s_{\text{guided}} = s_t(x) + w \nabla_x \log p_{\phi}(y|x, t)$$

CFG:

$$\varepsilon_{\text{cfg}} = (1+w)\varepsilon_{\theta}(x_t, t, y) - w\varepsilon_{\theta}(x_t, t, \emptyset)$$

FID:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Chapter 4 Questions and Answers

Question 1

Why does diffusion training reduce to an MSE loss even though the original objective is probabilistic?

Sample Answer:

The original objective is maximizing the ELBO. After modeling the forward process as Gaussian and fixing the reverse variance, the KL divergence terms reduce to differences between Gaussian means. By parameterizing the reverse mean using noise prediction, the optimization becomes equivalent to minimizing MSE between true and predicted noise.

Question 2

What is the intuition behind the score function in diffusion models?

Sample Answer:

The score is the gradient of the log-density. It points toward regions where probability increases most rapidly. In diffusion models, the score tells the sampler how to move noisy samples toward more likely data regions at the current noise level.

Question 3

Why is Gaussian noise added during reverse diffusion instead of using only deterministic denoising?

Sample Answer:

Without stochastic noise, reverse diffusion would behave like gradient ascent and collapse toward a few modes. Adding calibrated Gaussian noise preserves sampling diversity and allows the model to represent the full probability distribution rather than only the highest-density regions.

Question 4

Why is a noise-aware classifier necessary in classifier guidance?

Sample Answer:

The classifier operates on noisy intermediate states x_t during reverse diffusion. A classifier trained only on clean images may produce unstable or meaningless gradients at high noise

levels. A noise-aware classifier learns to classify corrupted samples and therefore provides guidance that matches the actual reverse trajectory.

Question 5

What is the core idea behind classifier-free guidance?

Sample Answer:

Classifier-free guidance trains a single diffusion model to produce both conditional and unconditional predictions. During training, conditions are randomly dropped so the model learns both behaviors. During sampling, the two predictions are combined to amplify conditional information without requiring an external classifier.

Question 6

Why does stronger guidance improve alignment but reduce diversity?

Sample Answer:

Stronger guidance amplifies updates toward condition-consistent regions of the distribution. This improves adherence to prompts or labels, but suppresses alternative plausible outputs. As a result, the generated distribution becomes narrower, reducing diversity and sometimes creating artifacts.

Question 7

What does FID measure conceptually?

Sample Answer:

FID compares the feature distributions of real and generated data using mean and covariance statistics in a deep feature space. Lower FID suggests that generated samples reproduce the semantic structure and variability of the real dataset more accurately.

Chapter 5 - Flow Matching

1. Big Picture of Flow Matching

Flow matching treats generation as a transport problem.

Goal:

transform a simple distribution such as Gaussian noise into the data distribution.

Instead of:

- learning latent posteriors like VAEs
- learning scores/noise like diffusion

Flow matching learns:

a time-dependent velocity field.

Main ODE:

$$dx/dt = v\theta(x,t)$$

If the velocity field is learned correctly:

particles starting from noise move toward the data distribution.

2. Wind Field Intuition

Flow matching can be understood using a wind-field analogy.

- x = particle location
- t = time
- $v\theta(x,t)$ = wind direction and speed

Sampling process:

1. initialize particles from noise
2. move them according to the learned wind field
3. final particle locations follow the data distribution

3. Main Difference from Diffusion

Diffusion:

- learns denoising/score functions
- tied to stochastic corruption processes

Flow matching:

- directly learns deterministic transport
- avoids explicit score estimation
- avoids reverse-time stochastic sampling

Diffusion learns:

how to denoise.

Flow matching learns:

how to move probability mass.

4. Main Difference from VAEs

VAEs require:

- encoder posterior $q\phi(z|x)$
- decoder likelihood $p\theta(x|z)$

Flow matching avoids:

- posterior approximation
- ELBO optimization
- KL divergence terms

Instead:

it directly learns a velocity field through supervised regression.

5. Flow Matching Training Setup

Training requires:

- start sample x_0 from base distribution p_0
- target sample x_1 from data distribution p_1
- random time t

Linear interpolation bridge:

$$x_t = (1-t)x_0 + tx_1$$

Target velocity:

$$u_t = x_1 - x_0$$

Training objective:

$$L = E ||v_\theta(x_t, t) - u_t||^2$$

Therefore:

flow matching becomes supervised vector-field regression.

6. Why the Velocity is Known

For linear interpolation:

$$x_t = (1-t)x_0 + tx_1$$

Differentiate with respect to time:

$$dx_t/dt = x_1 - x_0$$

Thus:

the target velocity is analytically known.

This is one of the biggest advantages of flow matching:
the supervision signal is explicit.

7. Continuity Equation

Flow matching models how an entire probability distribution evolves over time.

Continuity equation:

$$\partial p_t(x)/\partial t + \nabla \cdot (p_t(x)v(x,t)) = 0$$

Meaning:

probability mass is conserved.

Probability:

- does not disappear
- does not appear from nowhere
- only moves through space

8. Intuition Behind the Continuity Equation

If probability flows outward from a region:
density decreases there.

If probability flows inward:

density increases there.

Thus:

density changes are caused by probability transport.

9. Divergence Intuition

Divergence measures:
net outward flow from a region.

Positive divergence:
flow expands outward.

Negative divergence:
flow converges inward.

In flow matching:
divergence determines how probability density changes during transport.

10. Euler Integration

After training:
sampling solves the learned ODE numerically.

Euler update:

$$x_{n+1} = x_n + \Delta t v_{\theta}(x_n, t_n)$$

Interpretation:

- evaluate velocity
- move slightly in that direction
- repeat many times

11. Why Euler Works

Exact solution:

$$x(1) = x(0) + \int v_{\theta}(x(t), t) dt$$

Euler approximates the integral using Riemann sums.

Smaller step sizes:

- improve approximation
- produce smoother trajectories

12. Sampling in Flow Matching

Sampling process:

1. sample noise $x(0)$
2. integrate ODE from $t=0$ to $t=1$
3. output $x(1)$

Unlike diffusion:

sampling is deterministic ODE transport.

13. Importance of the Bridge

The interpolation bridge defines:
how intermediate states are constructed.

Linear bridge:

$$x_t = (1-t)x_0 + tx_1$$

Different bridges produce:

- different trajectories
- different supervision signals
- different learning difficulty

14. Why the Bridge Matters

The bridge changes:

- geometry of transport
- curvature of trajectories
- complexity of the velocity field

Thus:

the bridge is a modeling assumption,
not merely an implementation detail.

15. Optimal Transport (OT) Flow Matching

Regular flow matching:
pairs x_0 and x_1 randomly.

Optimal transport flow matching:
finds better pairings minimizing transport cost.

OT objective:

$$\pi^* = \operatorname{argmin} E ||x_0 - x_1||^2$$

Advantages:

- shorter trajectories
- smoother supervision
- easier learning
- fewer crossing paths

16. Latent Flow Matching

Instead of learning transport in pixel space:
transport is learned in latent space.

Encoder:

$$z = E\psi(x)$$

Decoder:

$$\hat{x} = D\psi(z)$$

Velocity field:

$$dz/dt = v\theta(z,t)$$

Advantages:

- lower dimensionality
- simpler geometry
- cheaper computation
- semantically cleaner space

17. Limitation of Latent Flow Matching

Performance depends heavily on the autoencoder quality.

If the encoder loses information:
the decoder becomes a bottleneck,
and generation fidelity decreases.

18. Conditional Flow Matching

Conditional flow matching:
velocity depends on a condition y .

ODE:

$$dx/dt = v\theta(x,t,y)$$

Applications:

- class conditional generation
- text conditioning
- controllable generation

19. Classifier-Free Guidance (CFG) in Flow Matching

The model learns:

- conditional velocity
- unconditional velocity

Guided velocity:

$$vcfg = v\theta(x,t,\emptyset) + s(v\theta(x,t,y) - v\theta(x,t,\emptyset))$$

where:

s = guidance scale

20. Interpretation of CFG

- unconditional branch provides realism
- conditional difference provides steering
- guidance scale controls condition strength

Large guidance:

- stronger condition alignment
- lower diversity

21. Classifier Guidance in Flow Matching

A separate classifier estimates class information.

Its gradient biases the velocity field toward desired conditions.

Important:

classifier must work on intermediate noisy states,
not only clean data.

22. Noise-Aware Classifiers

A standard clean-image classifier may fail on intermediate states.

Noise-aware classifiers are trained on noisy/intermediate samples.

Advantages:

- stable gradients
- meaningful conditioning
- improved guidance

23. Consistency Models

Consistency models learn mappings:

$$f_{\theta}(x_s, s) \approx f_{\theta}(x_t, t)$$

Goal:

multiple noisy versions map to the same canonical output.

Advantages:

- one-step generation
- very fast sampling

24. Why Consistency Models are Difficult

Consistency alone is insufficient.

A trivial constant function can satisfy consistency.

Problems:

- mode collapse
- oversmoothing
- degenerate solutions

Thus:

teacher guidance is usually needed.

25. Teacher-Student Consistency

A strong teacher model provides stable targets.

Usually:

- teacher = diffusion model
- student = consistency model

Teacher prevents collapse and encourages meaningful generation.

26. Transport-Based View of Generative Models

Modern generative models can all be viewed as transport systems.

VAE:

transport through latent bottlenecks.

Diffusion:

transport using denoising/score fields.

Flow matching:

transport using velocity fields.

Consistency models:

direct canonicalization transport.

27. Flow Matching vs Diffusion

Diffusion:

- learns scores/noise
- reverse stochastic process
- many denoising steps

Flow Matching:

- learns velocity fields
- deterministic ODE transport
- often fewer solver steps

28. Flow Matching vs VAEs

VAE:

- latent-variable probabilistic model
- requires ELBO optimization
- approximate posterior

Flow Matching:

- direct vector-field learning
- no posterior estimation
- deterministic transport

29. Time Embeddings

Flow models require timestep information.

Sinusoidal embeddings encode time t .

Time embedding allows:

- timestep-aware velocity prediction
- adaptive dynamics across trajectory

30. U-Net Backbone

Flow matching for images often uses U-Nets.

Reasons:

- captures spatial structure

- skip connections preserve details
- strong performance in generative modeling

31. Particle Trajectory Interpretation

Each particle follows:

$$dx/dt = v\theta(x,t)$$

Different particles collectively form the evolving probability distribution.

32. Vector Field Interpretation

$v\theta(x,t)$ is a vector.

Direction:
where to move.

Magnitude:
how fast to move.

33. 1D Normal Shift Example

Base:
 $N(0,1)$

Target:
 $N(\mu,1)$

Ideal velocity:
approximately constant drift μ .

Particles shift gradually toward the target distribution.

34. Bimodal and Trimodal Examples

In multimodal targets:
the velocity field becomes nonlinear.

Different regions are pushed toward different modes.

This demonstrates:
flow matching can learn complex probability geometries.

35. Main Advantages of Flow Matching

Advantages:

- direct supervision
- avoids score estimation
- avoids posterior approximation
- deterministic sampling
- often fewer steps
- flexible transport design

36. Main Limitations of Flow Matching

Limitations:

- depends strongly on bridge choice
- difficult velocity fields in high dimensions
- numerical solver accuracy matters
- latent quality affects LFM

37. Most Important Equations

Velocity field:

$$dx/dt = v\theta(x,t)$$

Linear bridge:

$$x_t = (1-t)x_0 + tx_1$$

Velocity target:

$$u_t = x_1 - x_0$$

Training loss:

$$L = E ||v\theta(x_t, t) - u_t||^2$$

Continuity equation:

$$\partial p_t / \partial t + \nabla \cdot (p_t v) = 0$$

Euler update:

$$x_{n+1} = x_n + \Delta t v\theta(x_n, t_n)$$

CFG:

$$vcfg = v\theta(x, t, \emptyset) + s(v\theta(x, t, y) - v\theta(x, t, \emptyset))$$

OT coupling:

$$\pi^* = \operatorname{argmin} E ||x_0 - x_1||^2$$

Chapter 5 Questions and Answers

Question 1

Why is flow matching considered a transport framework instead of only a regression method?

Sample Answer:

Although the training objective looks like supervised regression, the deeper object being learned is a velocity field that transports an entire probability distribution over time. The continuity equation governs how probability mass evolves, so the model is fundamentally learning transport dynamics rather than only minimizing MSE.

Question 2

Why does flow matching avoid score estimation unlike diffusion models?

Sample Answer:

Diffusion models indirectly learn the score function through denoising objectives. Flow matching instead constructs explicit interpolation paths where the target velocity is analytically known. Therefore, the model directly regresses velocities without estimating log-density gradients.

Question 3

Why is the interpolation bridge important in flow matching?

Sample Answer:

The bridge determines the geometry of intermediate states and the target velocities seen during training. Different bridges produce different trajectory shapes and learning difficulties, so the bridge acts as a modeling assumption rather than a trivial implementation detail.

Question 4

Why can latent flow matching outperform pixel-space flow matching?

Sample Answer:

Latent spaces are lower dimensional and often contain cleaner semantic structure than pixel space. This simplifies transport trajectories and reduces computational complexity.

However, performance depends heavily on the quality of the autoencoder representation.

Question 5

Why do optimal transport couplings improve flow matching?

Sample Answer:

Random pairings may create unnecessarily long or crossing trajectories. Optimal transport couplings create more coherent start-end pairings that minimize transport cost, leading to smoother trajectories and easier velocity-field learning.

Question 6

Why does stronger classifier-free guidance reduce diversity?

Sample Answer:

Stronger guidance amplifies the condition-specific component of the velocity field, forcing trajectories toward highly condition-consistent regions. This suppresses alternative plausible outputs and narrows the generated distribution.

Question 7

Why are consistency models attractive but difficult to train?

Sample Answer:

Consistency models aim for extremely fast one-step generation by mapping noisy versions of the same sample to a shared output. However, consistency alone can lead to trivial constant solutions or mode collapse, so stabilization methods such as teacher guidance are usually necessary.

Question 8

Why does Euler integration approximate the ODE solution correctly?

Sample Answer:

Euler integration comes from a first-order Taylor expansion of the true solution. Each update approximates the local velocity as constant over a very small interval. Repeating many small updates approximates the continuous-time integral defining the trajectory.

Chapter 6 - Transformers and Autoregressive Models

1. Autoregressive Modeling

Autoregressive models define the probability of a sequence as:

$$p(x_{1:T}) = \prod p(x_t | x_{<t})$$

Main idea:

predict the next token using previous tokens.

Training:

next-token prediction.

Generation:

predict token → append → repeat.

2. Why Autoregressive Modeling is Important

Instead of modeling an entire sequence at once,
the problem becomes a sequence of conditional prediction tasks.

This makes:

- training practical
- generation possible
- likelihood optimization tractable

3. Teacher Forcing

During training:

the model conditions on true previous tokens from the dataset.

This stabilizes optimization because errors do not accumulate during training.

4. Sampling / Generation

During inference:

the model conditions on its own generated outputs.

Process:

1. predict probability distribution
2. sample/select next token
3. append token
4. repeat

This creates autoregressive generation.

5. Exposure Bias

Training uses ground-truth history,
but generation uses model-generated history.

Therefore:
small generation mistakes can accumulate over time.

6. Recurrent Neural Networks (RNNs)

RNNs process sequences step-by-step.

Hidden state update:

$$h_t = f(h_{t-1}, x_t)$$

The hidden state acts as a compressed memory of previous tokens.

7. RNN Bottleneck

All historical information must be compressed into one hidden vector.

As sequences become long:

- information gets diluted
- long-range dependencies become difficult
- gradients become unstable

8. Backpropagation Through Time (BPTT)

RNNs are trained using BPTT.

The network is unrolled across time,

and gradients are propagated backward through all time steps.

9. Vanishing and Exploding Gradients

Repeated multiplication through time can:

- shrink gradients toward zero
- explode gradients to huge values

This makes long-range learning difficult.

10. LSTMs

LSTMs were designed to improve long-term memory retention.

Key idea:

introduce a memory cell with gates controlling information flow.

11. LSTM Gates

LSTMs contain:

- input gate
- forget gate
- output gate

These gates decide:

- what to write
- what to erase
- what to expose

12. Forget Gate Intuition

Forget gate:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

If $f_t \approx 1$:

keep memory.

If $f_t \approx 0$:

erase memory.

13. Input Gate Intuition

Input gate controls how much new information is written into memory.

$it \approx 1$:

strongly update memory.

$it \approx 0$:

ignore new information.

14. Output Gate Intuition

Output gate controls:

how much memory becomes visible as hidden state.

It determines what the model exposes externally.

15. Why LSTMs are Better than Vanilla RNNs

LSTMs create a more stable information path through time.

This helps:

- preserve long-term dependencies
- reduce vanishing gradients
- improve sequence learning

16. Transformer Motivation

Transformers were introduced to solve two major RNN/LSTM limitations:

1. sequential computation
2. weak long-range dependency learning

17. Main Idea of Transformers

Instead of compressing history into one hidden state,
Transformers allow tokens to directly interact with all relevant tokens using self-attention.

18. Self-Attention

Self-attention allows each token to:

- look at other tokens
- determine relevance
- aggregate useful information

This produces contextualized representations.

19. Contextual Representation

A token representation depends on surrounding context.

Example:

“bank” in:

- river bank
- bank account

has different contextual embeddings.

20. Query, Key, Value

Each token produces:

- query vector
- key vector
- value vector

Query:

what the token is looking for.

Key:

what the token offers.

Value:

the information payload.

21. Attention Score

Compatibility score:

$$s_{ij} = (q_i \cdot k_j) / \sqrt{d_k}$$

Large score:
strong relevance.

Small score:
weak relevance.

22. Why Divide by $\sqrt{d_k}$

Dot products grow with dimension.

Dividing by $\sqrt{d_k}$ stabilizes numerical scale
and prevents softmax saturation.

23. Softmax in Attention

Softmax converts raw scores into normalized attention weights:

$$\alpha_{ij} = \text{softmax}(s_{ij})$$

Properties:

- nonnegative
- sum to 1
- interpretable as importance weights

24. Weighted Sum of Values

Attention output:

$$y_i = \sum \alpha_{ij} v_j$$

Keys decide:
where to look.

Values decide:
what information to retrieve.

25. Multi-Head Attention

Transformers use multiple attention heads in parallel.

Different heads can specialize in:

- syntax
- long-range dependencies
- semantics
- positional patterns

26. Why Multi-Head Attention Helps

A single head captures one interaction pattern.

Multiple heads allow:
multiple relationships to be modeled simultaneously.

27. Transformer Block

A Transformer block usually contains:

1. multi-head attention
2. feed-forward network (FFN)
3. residual connections
4. layer normalization

28. Feed-Forward Network (FFN)

Attention mixes information across tokens.

FFN transforms information within each token independently.

29. Residual Connections

Residual connections preserve information flow:

$$x + F(x)$$

Advantages:

- easier optimization
- better gradient flow
- deeper networks

30. Layer Normalization

LayerNorm stabilizes training by normalizing token features.

Benefits:

- smoother optimization
- reduced internal covariate shift

31. Positional Encoding

Attention itself is permutation invariant.

Transformers therefore need positional information to understand token order.

32. Causal Masking

Decoder-only Transformers (GPT-style)
must not look at future tokens.

Causal masking enforces autoregressive constraints.

33. Why Causal Masking is Important

Without masking:
the model could directly see future answers during training.

This would violate autoregressive probability modeling.

34. Parallelism Advantage

Transformers process all sequence positions in parallel.

Unlike RNNs:
they do not require sequential hidden-state updates.

35. Long-Range Dependency Advantage

In Transformers:
any token can directly attend to any other token.

Path length between tokens becomes very short.

36. Complexity of Attention

Standard attention complexity:

$O(T^2)$

because every token compares with every other token.

37. Tokenization

Language models operate on tokens,
not raw words.

Tokenization determines:

- sequence length
- vocabulary structure
- representation granularity

38. Byte Pair Encoding (BPE)

BPE learns frequent subword units.

Advantages:

- handles rare words
- reduces vocabulary size
- balances word-level and character-level modeling

39. GPT-Style Models

GPT models are:
decoder-only Transformers trained autoregressively.

Objective:
next-token prediction.

40. Encoder vs Decoder Transformers

Encoder-only:
bidirectional understanding (BERT).

Decoder-only:
autoregressive generation (GPT).

Encoder-decoder:
sequence-to-sequence tasks (translation).

41. Why Transformers Became Dominant

Transformers became dominant because they:

- scale efficiently
- parallelize well
- model long-range dependencies strongly
- perform well on large datasets

42. Main Weakness of Transformers

Standard self-attention requires $O(T^2)$ memory and computation.

Very long sequences become expensive.

43. Key Intuition of Attention

Attention behaves like learned retrieval.

A token asks:

“What information do I need?”

Other tokens answer:

“Here is the information I contain.”

44. Most Important Equations

Autoregressive factorization:

$$p(x_{1:T}) = \prod p(x_t | x_{<t})$$

RNN update:

$$h_t = f(h_{t-1}, x_t)$$

Attention score:

$$s_{ij} = (q_i \cdot k_j) / \sqrt{d_k}$$

Softmax:

$$\alpha_{ij} = \text{softmax}(s_{ij})$$

Attention output:

$$y_i = \sum \alpha_{ij} v_j$$

LSTM memory:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Chapter 6 Questions and Answers

Question 1

Why is autoregressive modeling both a probabilistic framework and a training strategy?

Sample Answer:

It is probabilistic because it factorizes the joint probability of a sequence into conditional distributions over tokens. It is also a training strategy because maximizing likelihood becomes equivalent to repeatedly predicting the next token from the previous context.

Question 2

Why do RNNs struggle with very long-range dependencies?

Sample Answer:

RNNs compress all past information into a single evolving hidden state. As sequences become longer, information and gradients must travel through many sequential updates, causing vanishing gradients and difficulty preserving long-term information.

Question 3

Why are LSTMs more effective than vanilla RNNs?

Sample Answer:

LSTMs introduce gated memory mechanisms that regulate writing, forgetting, and exposing information. This creates more stable gradient flow and allows important information to persist across longer time spans.

Question 4

Why are separate query, key, and value projections necessary in self-attention?

Sample Answer:

Queries define what information a token is searching for, keys define how tokens advertise relevance, and values define the content being retrieved. Separating these roles gives the model greater flexibility and expressive power.

Question 5

Why does attention use a weighted sum of value vectors instead of directly averaging embeddings?

Sample Answer:

Different tokens contribute different levels of relevance to the current token. The weighted sum allows the model to emphasize important information while suppressing irrelevant information.

Question 6

Why is softmax necessary in attention?

Sample Answer:

Softmax converts arbitrary scores into normalized nonnegative weights that sum to one. This allows attention outputs to behave like stable weighted averages and gives scores an interpretable importance meaning.

Question 7

Why is causal masking essential in GPT-style Transformers?

Sample Answer:

Causal masking prevents tokens from attending to future positions during training. Without masking, the model could directly observe future answers, violating the autoregressive probability structure required for generation.

Question 8

Why can Transformers model long-range dependencies better than RNNs?

Sample Answer:

Transformers allow direct token-to-token interactions through self-attention. Any token can access another token in a single step, unlike RNNs where information must propagate sequentially through many intermediate states.

Question 9

Why does multi-head attention improve representational power?

Sample Answer:

Different heads can specialize in different relationships simultaneously, such as syntax, semantics, or positional structure. This creates richer contextual representations than a single attention map.

Question 10

Why are Transformers highly parallelizable compared to RNNs?

Sample Answer:

Transformers compute all token interactions simultaneously using matrix operations, while RNNs require sequential hidden-state updates that depend on previous time steps.

ELE768 Generative AI Notes

May 11, 2026

Chapter 7 - Vision Transformers

1. Core Idea of Vision Transformers

Vision Transformers (ViTs) apply Transformer architectures to images.

Instead of using convolutional filters, ViTs treat an image as a sequence of visual tokens.

Main pipeline:

image $\rightarrow$ patches $\rightarrow$ tokens $\rightarrow$ Transformer encoder

Main idea:

image patches $\approx$ language tokens

This allows self-attention to model relationships between different image regions.

2. Why ViTs Matter

ViTs are important because they provide a token-based visual representation.

This is useful for:

- image classification,
- image representation learning,
- vision-language models,
- generative models,
- multimodal systems.

Modern systems such as CLIP, SigLIP, MAE, DiT, and other generative architectures use ViT-style ideas.

3. From Images to Patches

A normal image has shape:

$$x \in \mathbb{R}^{H \times W \times C}$$

where:

- H = height,
- W = width,
- C = number of channels.

The image is split into patches of size:

$$P \times P.$$

Number of patches:

$$N = \frac{H}{P} \frac{W}{P}.$$

Each patch becomes one visual token.

4. Patch Embedding

Each image patch is flattened and projected into an embedding vector:

$$z_i = Ex_i$$

where:

- x_i = flattened patch,
- E = learned linear projection,
- $z_i \in \mathbb{R}^D$ = patch embedding,
- D = embedding dimension.

The image becomes a sequence:

$$Z = [z_1, z_2, \dots, z_N].$$

Patch embedding is the bridge from images to Transformers.

5. Patch Embedding Using Convolution

Patch embedding can be implemented using a convolutional layer with:

$$\text{kernel size} = P$$

$$\text{stride} = P.$$

This does two things at once:

- splits the image into non-overlapping patches,
- projects each patch into a D -dimensional token.

Example for CIFAR-10:

$$H = W = 32, \quad P = 4.$$

Then:

$$N = \frac{32}{4} \frac{32}{4} = 64.$$

If $D = 192$, the token sequence shape is:

$$[B, 64, 192].$$

6. Positional Embeddings

Self-attention is permutation-invariant.

This means the Transformer does not automatically know where each patch came from.

Therefore, ViTs add positional embeddings:

$$z'_i = z_i + p_i$$

where:

- z_i = patch embedding,
- p_i = positional embedding.

Without positional embeddings, the model knows the patches but not their spatial locations.

Standard ViT usually uses learned absolute positional embeddings:

$$Z' = Z + P_{\text{pos}}.$$

7. Why Positional Embeddings Matter

Images have spatial structure.

The location of a patch matters because:

- nearby patches often belong to the same object,
- top/bottom and left/right relationships are meaningful,
- object layout affects recognition.

Positional embeddings inject this spatial information into the token sequence.

8. Class Token

For classification, ViT usually adds a learnable class token:

$$x_{\text{cls}}.$$

It is prepended to the patch sequence:

$$X_0 = [x_{\text{cls}}; z_1; z_2; \dots; z_N].$$

The class token is not an image patch.

It is a learned vector updated during training.

9. Purpose of the Class Token

The class token acts as a global summary token.

Through self-attention, it can attend to all patch tokens and collect image-level information.

After the final Transformer layer, the class token is used for classification.

If the final output is:

$$X_L = [x_{\text{cls}}^{(L)}; x_1^{(L)}; \dots; x_N^{(L)}],$$

then:

$$\text{logits} = Wx_{\text{cls}}^{(L)} + b.$$

10. Is the Class Token Mandatory?

No.

Alternatives include:

- mean pooling over patch tokens,
- attention pooling,
- using another task-specific pooling method.

Class token is common in ViT classification, but general image encoders may use other pooling strategies.

11. Transformer Encoder Block

A ViT uses standard Transformer encoder blocks.

Each block contains:

- LayerNorm,
- Multi-Head Self-Attention,
- residual connection,
- LayerNorm,
- MLP,
- residual connection.

Block equations:

$$X' = X + \text{MSA}(\text{LN}(X))$$

$$X_{\text{out}} = X' + \text{MLP}(\text{LN}(X')).$$

12. Self-Attention in ViT

Self-attention allows every patch token to interact with every other patch token.

Given token matrix X :

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V.$$

Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Interpretation:

- queries ask what information is needed,
- keys indicate relevance,
- values contain the information retrieved.

13. Why Self-Attention Helps Vision

Self-attention gives every patch a direct path to every other patch.

This helps model:

- global context,
- object layout,
- long-range dependencies,
- relationships between distant image regions.

CNNs build global context gradually through many layers.

ViTs can model global relationships directly.

14. Multi-Head Attention

ViTs use multiple attention heads.

Different heads can learn different relationships, such as:

- local texture,
- object shape,
- symmetry,
- long-range structure,
- semantic relationships.

Multi-head attention increases representational power.

15. MLP Block

After attention, each token passes through an MLP independently.

Typical form:

$$\text{MLP}(x) = W_2 \sigma(W_1 x)$$

where σ is usually GELU.

Attention mixes information across tokens.

MLP transforms each token representation individually.

16. Residual Connections

Residual connections add the input back to the block output:

$$x + F(x).$$

Benefits:

- preserve information,
- improve gradient flow,
- stabilize deep training.

17. Layer Normalization

LayerNorm normalizes token features.

Purpose:

- stabilizes training,
- improves optimization,
- helps deep Transformer blocks train reliably.

ViTs commonly use pre-norm structure:

$$X + \text{Block}(\text{LN}(X)).$$

18. ViT for Classification

For classification, ViT maps an image to class logits:

$$f_{\theta}(x) \in \mathbb{R}^K$$

where K is the number of classes.

Usually:

- use final class token,
- apply linear classification head,
- train using cross-entropy loss.

19. ViT as a General Image Encoder

A ViT can also output an image feature vector:

$$h(x) \in \mathbb{R}^D.$$

This representation can be used for:

- retrieval,
- clustering,
- transfer learning,
- contrastive learning,
- vision-language alignment.

This is why ViTs are useful beyond classification.

20. ViT vs CNN

CNNs:

- use local convolutional filters,
- have strong locality bias,
- are efficient for spatial patterns,
- build global context gradually.

ViTs:

- use patch tokens,
- model global relationships using attention,
- have weaker built-in locality bias,
- usually need more data or regularization.

Short comparison:

CNNs: strong local inductive bias

ViTs: flexible global attention

21. Why ViTs Need More Data

CNNs assume local spatial structure through convolution.

ViTs make fewer assumptions.

This gives flexibility, but also means:

- more data is often needed,
- stronger augmentation may be needed,
- regularization becomes important.

22. ViTs and Generative AI

ViTs matter for generative AI because they represent images as tokens.

This makes it easier to:

- combine image tokens with text tokens,
- add conditioning information,
- process patches using attention,
- scale the same architecture to large models.

Examples:

- CLIP and SigLIP use ViT-style image encoders,
- MAE uses masked image modeling,
- DiT uses Transformer blocks for diffusion,
- multimodal systems use visual tokens with language tokens.

23. Important ViT Variants

Important related models:

- **ViT**: original patch-based image Transformer.
- **DeiT**: data-efficient ViT with distillation.
- **Swin Transformer**: hierarchical Transformer with shifted windows.
- **MAE**: masked autoencoder for self-supervised ViT learning.
- **CLIP / SigLIP**: vision-language models using ViT encoders.
- **DiT**: diffusion Transformer for image generation.

24. Main Weakness of ViTs

The main weakness of standard ViTs is attention cost.

For N tokens, self-attention has complexity:

$$O(N^2).$$

Since every patch attends to every other patch, high-resolution images can become expensive.

This motivates:

- window attention,
- hierarchical ViTs,
- efficient attention variants,
- latent-space processing.

25. Most Important Equations

Number of patches:

$$N = \frac{H}{P} \frac{W}{P}$$

Patch embedding:

$$z_i = Ex_i$$

Add positional embedding:

$$z'_i = z_i + p_i$$

Class-token input:

$$X_0 = [x_{\text{cls}}; z_1; z_2; \dots; z_N]$$

Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Transformer block:

$$X' = X + \text{MSA}(\text{LN}(X))$$

$$X_{\text{out}} = X' + \text{MLP}(\text{LN}(X'))$$

Classification head:

$$\text{logits} = Wx_{\text{cls}}^{(L)} + b.$$

Chapter 7 Sample Questions and Answers

Question 1

Why does a Vision Transformer split an image into patches?

Sample Answer:

A Transformer expects a sequence of tokens as input. Images are originally 2D arrays, so ViT splits the image into fixed-size patches and treats each patch as a visual token. This converts the image into a sequence that can be processed by a standard Transformer encoder.

Question 2

Why are positional embeddings necessary in ViTs?

Sample Answer:

Self-attention is permutation-invariant, so the Transformer does not automatically know the spatial position of each patch. Positional embeddings tell the model where each patch came from in the image. Without them, the model would know which patches exist but not their layout.

Question 3

What is the role of the class token in ViT?

Sample Answer:

The class token is a learned vector prepended to the patch sequence. Through self-attention, it gathers information from all image patches. After the final Transformer layer, its representation acts as a global summary of the image and is passed to the classification head.

Question 4

How is patch embedding implemented using convolution?

Sample Answer:

Patch embedding can be implemented using a convolutional layer with kernel size equal to the patch size and stride equal to the patch size. The stride makes patches non-overlapping, and the convolutional filters project each patch into the embedding dimension. This simultaneously extracts patches and embeds them.

Question 5

Why can ViTs capture global context better than CNNs?

Sample Answer:

In ViTs, self-attention allows every patch token to directly attend to every other patch token. This gives the model immediate access to long-range relationships. CNNs use local filters, so global context is built gradually through many layers.

Question 6

What is the difference between attention and the MLP block in a Transformer encoder?

Sample Answer:

Attention mixes information across tokens, allowing patches to exchange information. The MLP block then transforms each token independently. So attention handles token-to-token interaction, while the MLP improves each token's representation separately.

Question 7

Why do ViTs often need more data than CNNs?

Sample Answer:

CNNs have strong built-in assumptions about images, such as locality and translation structure. ViTs have weaker inductive bias because they treat images as token sequences. This makes them more flexible, but they often need more data, augmentation, or regularization to learn visual structure effectively.

Question 8

How can the same ViT encoder be used for classification and general image encoding?

Sample Answer:

For classification, the final class token is passed through a linear head to produce class logits. For general encoding, the model outputs a feature vector, either from the class token or pooled patch tokens. This feature vector can be used for retrieval, contrastive learning, transfer learning, or vision-language alignment.

Question 9

What is the main computational weakness of standard ViTs?

Sample Answer:

The main weakness is the quadratic cost of self-attention. If there are N patch tokens, attention compares every token with every other token, giving complexity $O(N^2)$. This becomes expensive for high-resolution images.

Question 10

Why are ViTs important for modern generative and multimodal models?

Sample Answer:

ViTs turn images into token sequences. This makes image representations compatible with Transformer-style processing and easier to combine with text tokens, time embeddings, and conditioning signals. This is why ViT-style encoders and backbones appear in CLIP, SigLIP, MAE, DiT, and other generative or multimodal systems.

Chapter 8 - Vision-Language Models

1. Core Idea of Vision-Language Models

Vision-Language Models (VLMs) connect visual information with language.

They learn relationships between:

- images,
- captions,
- prompts,
- descriptions,
- instructions.

Instead of only predicting fixed labels, VLMs learn semantic alignment between images and text.

2. Why VLMs Matter

Classic vision models are usually trained on fixed label sets.

VLMs use language supervision, which allows more flexible behavior.

They can support:

- image-to-text retrieval,
- text-to-image retrieval,
- zero-shot classification,
- image captioning,
- visual question answering,
- multimodal instruction following.

Main idea:

language provides flexible supervision
instead of only fixed class labels.

3. Training Data

Most VLMs are trained on paired image-text data:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

where:

- x_i = image,

- y_i = associated text.

At large scale, pairs often come from the web.

Advantages:

- broad visual-language knowledge,
- diverse concepts,
- scalable supervision.

Problems:

- noisy captions,
- incomplete descriptions,
- weak image-text correspondence,
- bias,
- incorrect metadata.

4. Two Main VLM Families

There are two dominant architecture families:

- **Dual-encoder contrastive models**
- **Fusion / encoder-decoder models**

Short distinction:

dual encoders compare modalities

fusion models combine modalities

5. Dual-Encoder VLMs

Dual-encoder models use two separate encoders:

$$v_i = f_{\theta}(x_i)$$

$$t_i = g_{\phi}(y_i)$$

where:

- f_{θ} = image encoder,
- g_{ϕ} = text encoder,
- v_i = image embedding,
- t_i = text embedding.

The goal is to map images and text into a shared embedding space.

6. Normalized Embeddings

Embeddings are usually L2-normalized:

$$\hat{v}_i = \frac{v_i}{\|v_i\|}$$
$$\hat{t}_i = \frac{t_i}{\|t_i\|}.$$

This makes dot product similarity equivalent to cosine similarity.

7. Image-Text Similarity

Similarity between image i and text j :

$$s_{ij} = \frac{\hat{v}_i^T \hat{t}_j}{\tau}$$

where:

- s_{ij} = similarity score,
- τ = temperature,
- smaller τ makes the softmax sharper.

8. Why Dual Encoders Are Efficient

Image and text embeddings are computed separately.

This means:

- image embeddings can be cached,
- text embeddings can be cached,
- retrieval becomes fast,
- nearest-neighbor search can be used.

Dual encoders are strong for:

- retrieval,
- zero-shot classification,
- large-scale search.

9. Fusion / Encoder-Decoder VLMs

Fusion models combine image and text tokens inside the model.

Instead of only comparing global embeddings, they allow cross-modal interaction.

Usually:

- image encoder produces visual tokens,
- text decoder processes language tokens,
- cross-attention connects text and image information.

Fusion models are useful for:

- captioning,
- VQA,
- instruction following,
- multimodal reasoning.

10. Conditional Generation Objective

Fusion models often use autoregressive generation:

$$\log p_{\psi}(y|x) = \sum_{t=1}^T \log p_{\psi}(y_t|y_{<t}, x).$$

Meaning:

the model generates text token by token, conditioned on the image and previous tokens.

11. CLIP Core Idea

CLIP stands for Contrastive Language-Image Pretraining.

It is a dual-encoder model.

CLIP trains:

- an image encoder,
- a text encoder,

so that matched image-text pairs have high similarity and mismatched pairs have low similarity.

12. CLIP Similarity Matrix

Given B matched image-text pairs:

$$(x_1, y_1), (x_2, y_2), \dots, (x_B, y_B),$$

compute all pairwise similarities:

$$s_{ij} = \frac{\hat{v}_i^T \hat{t}_j}{\tau}.$$

This forms a matrix:

$$S \in \mathbb{R}^{B \times B}.$$

Diagonal entries:

$$s_{ii}$$

are correct image-text matches.

Off-diagonal entries are mismatches and act as negatives.

13. Image-to-Text Contrastive Loss

For each image i , the model chooses the correct text among all batch texts:

$$p_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^B \exp(s_{ik})}.$$

The image-to-text loss is:

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{B} \sum_{i=1}^B \log p_{ii}.$$

This is cross-entropy over text candidates.

14. Text-to-Image Contrastive Loss

Similarly, text-to-image matching uses the transpose of the similarity matrix:

$$\mathcal{L}_{T \rightarrow I}.$$

This asks:

given a text, which image in the batch is the correct match?

15. Symmetric CLIP Loss

The total CLIP loss is:

$$\mathcal{L} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I}).$$

This trains alignment in both directions.

16. Pull-Push Interpretation

CLIP training has two forces:

pull matched pairs together

push mismatched pairs apart.

For each image:

- its paired caption is the positive,
- other captions in the batch are negatives.

Hard negatives are mismatched examples that currently look similar to the model.

They contribute more strongly to the loss.

17. Temperature

Temperature controls softmax sharpness:

$$s_{ij} = \frac{\hat{v}_i^T \hat{t}_j}{\tau}.$$

Small τ :

- sharper softmax,
- stronger focus on hard negatives,
- larger gradient emphasis on difficult mismatches.

Large τ :

- softer distribution,
- less aggressive separation.

CLIP often learns a logit scale, equivalent to learning $1/\tau$.

18. Zero-Shot Classification with CLIP

CLIP can classify images without training a classifier head.

For each class c , create a prompt:

“a photo of a c ”.

Encode it to get a class prototype:

$$w_c = g_\phi(\text{prompt}(c)).$$

Given image embedding v , predict:

$$\hat{c} = \arg \max_c v^T w_c.$$

This enables open-vocabulary recognition.

19. Prompt Engineering

Zero-shot CLIP depends on text prompts.

Different prompts can give different results.

Example templates:

- “a photo of a {class}”
- “an image of a {class}”
- “a close-up photo of a {class}”

Prompt ensembling averages multiple prompt embeddings to improve robustness.

20. Cross-Modal Retrieval

Because images and text share an embedding space, retrieval is natural.

Text-to-image retrieval:

retrieve images with largest $v_i^T t_q$.

Image-to-text retrieval:

retrieve texts with largest $v_q^T t_j$.

This is efficient because embeddings can be precomputed.

21. Cross-Attention in Fusion Models

Fusion models use cross-attention to connect text and image tokens.

Let:

- H = text decoder states,
- M = visual memory from image encoder.

Then:

$$Q = HW_Q$$

$$K = MW_K$$

$$V = MW_V.$$

Cross-attention:

$$\text{CrossAttn}(H, M) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

Text tokens query the visual memory.

22. Meaning of Cross-Attention

In cross-attention:

- queries come from text,
- keys come from image tokens,
- values come from image tokens.

Interpretation:

each text token retrieves relevant visual evidence from the image representation.

This is why fusion models are better for captioning and VQA.

23. Dual Encoder vs Fusion Model

Dual encoder:

- compares image and text embeddings,
- efficient,
- cacheable,
- good for retrieval and zero-shot classification.

Fusion model:

- combines image and text tokens,
- more expressive,
- better for generation and reasoning,
- more expensive.

Main tradeoff:

$$\text{efficiency} \leftrightarrow \text{expressiveness}.$$

24. VLM Evaluation

VLMs should be evaluated across multiple axes.

Important metrics:

- retrieval Recall@K,
- zero-shot classification accuracy,
- robustness,
- bias and fairness,
- grounding ability,
- hallucination rate,
- latency,
- memory,
- energy use.

No single benchmark fully captures VLM quality.

25. Recall@K

Recall@K is common for retrieval.

It asks:

Is the correct match in the top K retrieved results?

Higher Recall@K means better retrieval performance.

26. VLM Limitations

Main limitations:

- noisy web-scale data,
- dataset bias,
- spurious correlations,
- weak fine-grained grounding,
- difficulty with counting and spatial relations,
- hallucination in generative VLMs,
- domain shift.

Dual encoders often learn global alignment but may miss local details.

Fusion models are more expressive but can hallucinate unsupported information.

27. Beyond CLIP

Important related systems:

- **ALIGN**: large-scale dual-encoder training on noisy web data.
- **SigLIP**: uses pairwise sigmoid loss instead of CLIP’s batch softmax.
- **OpenCLIP**: open CLIP-style implementations and checkpoints.
- **BLIP-2**: connects vision encoders to language models.
- **Flamingo**: multimodal generation and instruction following.
- **LLaVA**: vision-language instruction tuning.
- **OWL-ViT**: open-vocabulary detection.
- **Grounding DINO**: vision-language grounding and detection.

28. Main Idea of VLMs

VLMs connect vision and language.

CLIP-like models learn:

shared image-text embedding space.

Fusion models learn:

joint image-text reasoning.

Both are central to modern multimodal AI.

29. Most Important Equations

Dual-encoder embeddings:

$$v_i = f_\theta(x_i), \quad t_i = g_\phi(y_i)$$

Normalized embeddings:

$$\hat{v}_i = \frac{v_i}{\|v_i\|}, \quad \hat{t}_i = \frac{t_i}{\|t_i\|}$$

Similarity:

$$s_{ij} = \frac{\hat{v}_i^T \hat{t}_j}{\tau}$$

Softmax probability:

$$p_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^B \exp(s_{ik})}$$

Image-to-text loss:

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{B} \sum_{i=1}^B \log p_{ii}$$

Symmetric CLIP loss:

$$\mathcal{L} = \frac{1}{2}(\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I})$$

Zero-shot prediction:

$$\hat{c} = \arg \max_c v^T w_c$$

Fusion objective:

$$\log p_\psi(y|x) = \sum_{t=1}^T \log p_\psi(y_t|y_{<t}, x)$$

Cross-attention:

$$\text{CrossAttn}(H, M) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Chapter 8 Sample Questions and Answers

Question 1

What is the main idea of a Vision-Language Model?

Sample Answer:

A Vision-Language Model learns to connect images and text. Instead of only recognizing fixed labels, it learns semantic alignment between visual content and language. This allows tasks such as retrieval, captioning, zero-shot classification, VQA, and multimodal instruction following.

Question 2

What is the difference between a dual-encoder VLM and a fusion VLM?

Sample Answer:

A dual-encoder VLM encodes images and text separately, then compares their embeddings. This is efficient and useful for retrieval and zero-shot classification. A fusion VLM combines image and text tokens inside the model, often using cross-attention, which makes it stronger for captioning, VQA, and multimodal reasoning.

Question 3

Why does CLIP use a similarity matrix during training?

Sample Answer:

For a batch of matched image-text pairs, CLIP compares every image with every text. The diagonal entries are correct matches, while off-diagonal entries are mismatches. This lets the model train using contrastive learning by pulling correct pairs together and pushing incorrect pairs apart.

Question 4

What is the pull-push interpretation of CLIP?

Sample Answer:

CLIP pulls matched image-text pairs closer in the shared embedding space and pushes mismatched pairs farther apart. The matched caption is the positive example, while other captions in the batch act as negatives. Hard negatives influence the loss more strongly.

Question 5

What does the temperature parameter do in CLIP?

Sample Answer:

The temperature controls the sharpness of the softmax over similarities. A smaller temperature makes the distribution sharper and forces the model to focus more on hard negatives. A larger temperature gives a softer distribution and weaker separation pressure.

Question 6

How does CLIP perform zero-shot classification?

Sample Answer:

CLIP creates text prompts for each class, such as “a photo of a cat,” and encodes them into text embeddings. It then compares the image embedding with each class prompt embedding and chooses the class with the highest similarity. This allows classification using arbitrary text labels.

Question 7

Why are dual encoders efficient for retrieval?

Sample Answer:

Dual encoders compute image and text embeddings separately. These embeddings can be cached and compared using dot products or nearest-neighbor search. This makes large-scale retrieval much faster than models that must jointly process every image-text pair.

Question 8

Why are fusion models better for captioning and VQA?

Sample Answer:

Fusion models allow text tokens to attend directly to image tokens through cross-attention. This lets the model retrieve visual evidence while generating each word. This token-level interaction is more expressive than simply comparing global image and text embeddings.

Question 9

What is hallucination in generative VLMs?

Sample Answer:

Hallucination occurs when a model generates plausible text that is not supported by the image. For example, it may describe objects, actions, or details that are not actually visible. This is a common limitation of fusion and generative VLMs.

Question 10

Why should VLMs be evaluated using multiple metrics?

Sample Answer:

VLMs perform many different tasks, so one metric is not enough. Retrieval needs Recall@K, classification needs accuracy, generative tasks need faithfulness and hallucination checks, and deployment needs latency, memory, and energy evaluation. Robustness, bias, and grounding are also important.

Chapter 9 - Training and Adapting Vision-Language Models

1. Core Idea of VLM Adaptation

Adapting a VLM means changing how it behaves for a specific task, domain, or user requirement.

Adaptation methods range from cheap inference-time methods to expensive training-based methods.

Main ladder:

prompting $\rightarrow$ RAG $\rightarrow$ SFT $\rightarrow$ LoRA/QLoRA $\rightarrow$ DPO $\rightarrow$ RL

2. Adaptation Ladder

From cheapest to most expensive:

- zero-shot prompting,
- few-shot prompting,
- retrieval-augmented generation,
- supervised fine-tuning,
- LoRA / QLoRA,
- preference optimization,
- RL-based adaptation.

Main distinction:

prompting and RAG do not update weights

SFT, LoRA, DPO, and RL update model parameters.

3. VLM Policy

A generative VLM defines:

$$p_{\theta}(y|x, v)$$

where:

- x = text prompt,
- v = visual input,
- $y = (y_1, \dots, y_T)$ = generated answer,

- θ = model parameters.

Autoregressive factorization:

$$p_{\theta}(y|x, v) = \prod_{t=1}^T p_{\theta}(y_t|y_{<t}, x, v).$$

Meaning:

the answer is generated one token at a time.

4. Zero-Shot Prompting

Zero-shot prompting uses the pretrained model directly.

The model receives:

- image,
- instruction,
- no examples,
- no training updates.

No parameter update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

does not happen.

Prediction:

$$\hat{y} \approx \arg \max_y p_{\theta}(y|x, v).$$

5. Why Zero-Shot Works

Zero-shot works because the model already learned broad visual-language knowledge during pretraining.

It is useful for:

- quick baselines,
- simple tasks,
- tasks close to pretraining data.

Limitations:

- no new knowledge,
- prompt sensitivity,
- weak calibration,
- poor fine-grained domain recognition.

6. Few-Shot Prompting

Few-shot prompting adds worked examples into the context.

Demonstration set:

$$D_{\text{ctx}} = \{(x_j, v_j, y_j)\}_{j=1}^m.$$

The model answers using:

$$p_{\theta}(y|x, v, D_{\text{ctx}}).$$

Weights remain frozen.

7. Why Few-Shot Prompting Helps

Few-shot examples can:

- show the desired output format,
- clarify the task,
- restrict the label space,
- provide examples for comparison,
- guide style and structure.

However:

adaptation disappears when the context is removed.

8. Retrieval-Augmented Generation

RAG adds external retrieved information to the prompt.

Main structure:

retriever $\rightarrow$ evidence

generator $\rightarrow$ answer

The model does not need to store all knowledge in its parameters.

9. RAG Probabilistic View

A simple RAG formulation:

$$p_{\theta, \eta}(y|x, v) = \sum_{z \in Z} p_{\eta}(z|q(x, v)) p_{\theta}(y|x, v, z)$$

where:

- z = retrieved document,
- Z = document collection,
- $p_\eta(z|q(x, v))$ = retriever,
- $p_\theta(y|x, v, z)$ = generator conditioned on evidence.

In practice, use top- k retrieved documents.

10. RAG Strengths and Weaknesses

RAG is useful when:

- external knowledge is needed,
- domain facts change,
- model knowledge is incomplete.

Weaknesses:

- retrieval quality is a bottleneck,
- irrelevant evidence can hurt,
- wrong evidence can create false confidence,
- long contexts increase cost.

11. Supervised Fine-Tuning

Supervised fine-tuning updates model parameters using labeled examples.

Dataset:

$$\mathcal{D} = \{(x_i, v_i, y_i)\}_{i=1}^N.$$

Goal:

$$\max_{\theta} p_{\theta}(y_i | x_i, v_i).$$

Equivalent loss:

$$\mathcal{L}_{SFT} = - \sum_{i=1}^N \sum_{t=1}^{T_i} \log p_{\theta}(y_{i,t} | y_{i,<t}, x_i, v_i).$$

This is token-level cross-entropy.

12. What SFT Teaches

SFT teaches the model to imitate desired outputs.

At each token:

- model predicts next-token distribution,
- correct token probability is increased,
- parameters are updated by gradient descent.

Unlike prompting, SFT creates persistent adaptation.

13. Assistant-Token Loss

In chat-style training, the full sequence contains:

- system tokens,
- user tokens,
- image tokens,
- assistant tokens.

Usually, we train only on assistant response tokens.

Masked loss:

$$\mathcal{L} = - \sum_{t=1}^T m_t \log p_{\theta}(y_t | y_{<t}, x, v).$$

where:

$$m_t = 1$$

for assistant tokens, and

$$m_t = 0$$

for ignored tokens.

14. LoRA

LoRA means Low-Rank Adaptation.

Instead of updating a full weight matrix W , LoRA learns a low-rank update:

$$W' = W + \Delta W$$

$$\Delta W = BA.$$

If:

$$W \in \mathbb{R}^{d \times k},$$

then:

$$A \in \mathbb{R}^{r \times k}, \quad B \in \mathbb{R}^{d \times r},$$

where:

$$r \ll d, k.$$

15. Why LoRA is Efficient

LoRA is efficient because:

- base model is frozen,
- only adapter matrices are trained,
- memory use is lower,
- adapters can be stored separately,
- multiple task adapters can share the same base model.

LoRA often targets:

- attention projections,
- MLP projections.

16. QLoRA

QLoRA combines quantization and LoRA.

Main structure:

frozen quantized base model + trainable LoRA adapters.

The base model is often loaded in 4-bit precision.

The adapters are trained normally.

QLoRA makes fine-tuning large models practical on limited GPU memory.

17. Important Note About LoRA and QLoRA

LoRA and QLoRA do not change the task loss.

Usually, the loss is still SFT cross-entropy.

They only change:

- which parameters are trained,
- how the update is parameterized,
- how much memory is needed.

18. Preference Data

Preference optimization uses examples:

$$(x, v, y^+, y^-)$$

where:

- x = prompt,
- v = image,
- y^+ = preferred response,
- y^- = rejected response.

Preferred responses may be:

- more accurate,
- better formatted,
- less overconfident,
- more faithful to the image,
- better calibrated.

19. Direct Preference Optimization

DPO trains the model to prefer chosen responses over rejected responses.

It compares the current policy to a frozen reference policy.

Preference margin:

$$\Delta = \beta \left[\log \frac{\pi_{\theta}(y^+|x, v)}{\pi_{\text{ref}}(y^+|x, v)} - \log \frac{\pi_{\theta}(y^-|x, v)}{\pi_{\text{ref}}(y^-|x, v)} \right].$$

DPO loss:

$$\mathcal{L}_{DPO} = -\log \sigma(\Delta).$$

20. DPO Intuition

DPO does three things:

- increases probability of preferred response,
- decreases probability of rejected response,
- prevents excessive drift from the reference model.

DPO is simpler than full RLHF because it avoids:

- explicit reward model training,
- rollout sampling,
- advantage estimation.

21. Reward Modeling

RLHF-style training often uses a reward model:

$$r_\phi(x, v, y).$$

It maps a prompt, image, and answer to a scalar reward.

Pairwise reward-model loss:

$$\mathcal{L}_{RM} = -\log \sigma \left(r_\phi(x, v, y^+) - r_\phi(x, v, y^-) \right).$$

The reward model learns to score preferred answers higher than rejected answers.

22. RL-Based Adaptation

RL-based adaptation directly optimizes the VLM policy using rewards.

A KL-regularized objective:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}} [r(x, v, y) - \lambda D_{KL}(\pi_{\theta}(\cdot|x, v) \parallel \pi_{\text{ref}}(\cdot|x, v))].$$

Reward encourages better answers.

KL prevents the model from drifting too far from the reference model.

23. RL Notation for VLMs

In VLM alignment:

- **State:** image, prompt, generated prefix.
- **Action:** next generated token.
- **Policy:** VLM next-token distribution.
- **Trajectory:** full generated answer.
- **Reward:** scalar score for the answer.
- **Advantage:** how much better output was than expected.

RLHF is token-level policy optimization over multimodal prompts.

24. PPO

PPO avoids overly large policy updates.

Probability ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

Clipped objective:

$$\mathcal{L}_{PPO} = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

Main idea:

- increase probability of good actions,
- decrease probability of bad actions,
- prevent overly aggressive updates.

25. RLOO

RLOO means REINFORCE Leave-One-Out.

It generates multiple responses for the same prompt.

Then it estimates a baseline by comparing each response to the others.

Purpose:

- reduce variance,
- improve policy-gradient estimates,
- avoid needing a separate value model.

26. Choosing an Adaptation Method

Use zero-shot when:

- you need a fast baseline,
- no training data is available.

Use few-shot when:

- format matters,
- examples help clarify the task.

Use RAG when:

- external knowledge is needed.

Use SFT when:

- labeled examples are available,
- behavior should be learned permanently.

Use LoRA/QLoRA when:

- full fine-tuning is too expensive.

Use DPO when:

- preference pairs are available.

Use RL-based methods when:

- explicit rewards are available,
- the extra cost is justified.

27. Main Tradeoff

Prompting and RAG:

cheap and temporary

SFT, LoRA, DPO, RL:

more expensive but persistent

Main adaptation tradeoff:

cost $\leftrightarrow$ strength of adaptation.

28. Most Important Equations

VLM policy:

$$p_{\theta}(y|x, v)$$

Autoregressive factorization:

$$p_{\theta}(y|x, v) = \prod_{t=1}^T p_{\theta}(y_t|y_{<t}, x, v)$$

Few-shot conditioning:

$$p_{\theta}(y|x, v, D_{\text{ctx}})$$

RAG:

$$p_{\theta, \eta}(y|x, v) = \sum_{z \in Z} p_{\eta}(z|q(x, v)) p_{\theta}(y|x, v, z)$$

SFT loss:

$$\mathcal{L}_{SFT} = - \sum_{i=1}^N \sum_{t=1}^{T_i} \log p_{\theta}(y_{i,t}|y_{i,<t}, x_i, v_i)$$

Assistant-token loss:

$$\mathcal{L} = - \sum_{t=1}^T m_t \log p_{\theta}(y_t | y_{<t}, x, v)$$

LoRA:

$$W' = W + BA$$

DPO:

$$\mathcal{L}_{DPO} = -\log \sigma(\Delta)$$

Reward-model loss:

$$\mathcal{L}_{RM} = -\log \sigma(r_{\phi}(x, v, y^+) - r_{\phi}(x, v, y^-))$$

KL-regularized RL objective:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}} [r(x, v, y) - \lambda D_{KL}(\pi_{\theta}(\cdot | x, v) \| \pi_{\text{ref}}(\cdot | x, v))]$$

Chapter 9 Sample Questions and Answers

Question 1

What is the main difference between prompting/RAG and fine-tuning methods?

Sample Answer:

Prompting and RAG do not update model weights. They only change the input context. Fine-tuning methods such as SFT, LoRA, DPO, and RL update the model policy itself, so the adaptation is stored in the parameters.

Question 2

Why is zero-shot prompting useful even if it is weak?

Sample Answer:

Zero-shot prompting is useful because it is cheap, fast, and requires no training data. It provides a baseline for later adaptation methods. If zero-shot performance is already good, more expensive methods may not be necessary.

Question 3

Why can few-shot prompting change model behavior without changing weights?

Sample Answer:

Few-shot prompting changes the context that the model attends to during the forward pass. The demonstrations influence hidden states, attention patterns, and logits. The model parameters remain fixed, but the conditioning information changes.

Question 4

What is the main idea of RAG?

Sample Answer:

RAG retrieves external evidence and places it into the model context before generation. The retriever finds relevant documents, and the generator answers using those documents. This helps when the model needs domain-specific or updated knowledge.

Question 5

Why can RAG fail?

Sample Answer:

RAG can fail if retrieval returns irrelevant or incorrect evidence. The generator may trust the retrieved context and produce a confident but wrong answer. Long or noisy retrieved contexts can also distract the model and increase inference cost.

Question 6

What does SFT optimize?

Sample Answer:

SFT minimizes token-level negative log-likelihood of the desired answer. At each step, the model is trained to assign higher probability to the correct next token, conditioned on the prompt, image, and previous answer tokens.

Question 7

Why use assistant-token loss?

Sample Answer:

In chat fine-tuning, we usually want the model to learn how to answer, not to imitate the user prompt or system message. Assistant-token loss masks out non-assistant tokens and computes loss only on the assistant response.

Question 8

What is the main idea behind LoRA?

Sample Answer:

LoRA freezes the original weight matrix and learns a small low-rank update. Instead of training all weights, it trains small matrices A and B , where $\Delta W = BA$. This makes adaptation much cheaper in memory and compute.

Question 9

How is QLoRA different from LoRA?

Sample Answer:

QLoRA uses a quantized frozen base model, often in 4-bit precision, together with trainable LoRA adapters. This reduces memory use further while keeping the adaptation mechanism similar to LoRA.

Question 10

How is DPO different from SFT?

Sample Answer:

SFT teaches the model to imitate a target answer. DPO teaches the model to prefer one answer over another. It uses chosen and rejected responses and updates the policy so that chosen responses become more likely relative to rejected responses and relative to a reference model.

Question 11

Why does RL-based adaptation include KL regularization?

Sample Answer:

KL regularization prevents the adapted model from drifting too far from the reference model. Without it, the model may exploit weaknesses in the reward function and produce unnatural or unsafe outputs. The KL term keeps optimization stable.

Question 12

When should LoRA or QLoRA be preferred over full fine-tuning?

Sample Answer:

LoRA or QLoRA should be preferred when full fine-tuning is too expensive in memory or compute. They allow practical specialization by training only small adapter parameters while keeping the base model frozen.

Chapter 10 - Quantization

1. Core Idea of Quantization

Quantization reduces numerical precision in neural networks.

Instead of storing values in FP32 or FP16, we store or compute them using lower-bit formats such as:

INT8, INT4, INT2

Main idea:

high precision values $\rightarrow$ low precision approximations

This makes models cheaper to store and run.

2. Why Quantization is Useful

Quantization helps reduce:

- memory footprint,
- memory bandwidth,
- inference latency,
- energy consumption,
- deployment cost.

If a layer has n_ℓ parameters and each parameter uses b_ℓ bits, storage is:

$$M_\ell = n_\ell b_\ell.$$

So reducing bit-width directly reduces storage.

Example:

$$\text{FP32} \rightarrow \text{INT8}$$

gives ideal $4\times$ parameter compression.

$$\text{FP32} \rightarrow \text{INT4}$$

gives ideal $8\times$ parameter compression.

3. Basic Quantization Process

Quantization maps a real value x to an integer code q , then reconstructs an approximation $\hat{x}$.

$$x \rightarrow q \rightarrow \hat{x}$$

The stored value is q .

The reconstructed low-precision approximation is $\hat{x}$.

4. Affine Quantization

A standard affine quantizer is:

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right) + z, q_{\min}, q_{\max} \right).$$

Reconstruction:

$$\hat{x} = s(q - z).$$

Where:

- x = original real value,

- q = stored integer code,
- s = scale or step size,
- z = zero-point,
- $q_{\min}, q_{\max}$ = allowed integer range.

5. Scale

The scale s controls the spacing between quantization levels.

Small s :

fine grid

Large s :

coarse grid

The scale is one of the most important quantization parameters.

6. Zero-Point

The zero-point z shifts the integer grid.

It allows real zero to be represented exactly in affine quantization.

In symmetric quantization:

$$z = 0.$$

7. Symmetric Quantization

In symmetric quantization:

$$z = 0.$$

The quantization rule becomes:

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right), -Q, Q \right).$$

For signed b -bit quantization:

$$Q = 2^{b-1} - 1.$$

Reconstruction:

$$\hat{x} = sq.$$

A common scale is:

$$s = \frac{\max |x|}{Q}.$$

This makes the largest magnitude value fit into the integer range.

8. Range and Resolution Tradeoff

The scale s controls both:

- quantization resolution,
- representable range.

Representable values:

$$\{sq_{\min}, \dots, sq_{\max}\}.$$

Small scale:

better resolution but more clipping

Large scale:

larger range but more rounding error

This is the central quantization tradeoff.

9. Quantization Error

Quantization error:

$$e = x - \hat{x}.$$

Mean squared error:

$$\text{MSE} = \mathbb{E}[(x - \hat{x})^2].$$

Two main error sources:

- **rounding error:** value is rounded to nearest level,
- **clipping error:** value exceeds representable range.

10. Clipping

Clipping limits values to a fixed range before quantization.

If the clipping threshold is α :

$$x_{\text{clip}} = \text{clip}(x, -\alpha, \alpha).$$

Clipping can help because rare outliers no longer force the scale to become too large.

But excessive clipping removes useful signal.

11. Per-Tensor Quantization

Per-tensor quantization uses one scale for the whole tensor:

$$\hat{W} = s \cdot \text{round} \left(\frac{W}{s} \right).$$

Advantage:

simple and cheap

Disadvantage:

poor if channels have different ranges

12. Per-Channel Quantization

Per-channel quantization uses one scale per channel:

$$\hat{W}_{i,:} = s_i \cdot \text{round} \left(\frac{W_{i,:}}{s_i} \right).$$

This is usually more accurate because each channel gets its own dynamic range.

13. Group-Wise Quantization

Group-wise quantization divides channels into groups.

Each group has its own scale:

$$\hat{W}_{G_k} = s_k \cdot \text{round} \left(\frac{W_{G_k}}{s_k} \right).$$

This balances:

accuracy $\leftrightarrow$ metadata cost.

It is common in large transformers.

14. Three Phases of Quantization

Quantization methods are often grouped into three phases:

- **Pre-quantization**
- **During-training quantization**
- **Post-training quantization**

These phases all manage the same problem:

lower precision $\Rightarrow$ higher approximation risk.

15. Pre-Quantization

Pre-quantization happens before applying quantization.

It asks:

Which values must be represented, and where will quantization hurt most?

It includes:

- calibration,
- range estimation,
- outlier analysis,
- sensitivity analysis,
- choosing bit-widths,
- choosing quantization granularity,
- deciding which layers remain high precision.

16. Calibration

Calibration estimates tensor ranges using representative data.

For symmetric min-max calibration:

$$s = \frac{\max_{a \in C} |a|}{Q},$$

where C is the calibration set.

Percentile calibration:

$$s = \frac{\text{Percentile}_p(|a|)}{Q}.$$

Percentile calibration is often more robust to outliers than min-max calibration.

17. Sensitivity Analysis

Not all layers are equally sensitive to quantization.

If layer ℓ is quantized, the loss change can be approximated as:

$$\Delta \mathcal{L}_\ell \approx \frac{1}{2} \text{vec}(\Delta W_\ell)^T H_\ell \text{vec}(\Delta W_\ell).$$

Where:

- ΔW_ℓ = quantization perturbation,
- H_ℓ = Hessian for that layer.

Sensitive layers should often remain in higher precision.

18. Mixed Precision

Mixed precision assigns different bit-widths to different layers.

Example:

sensitive layers $\rightarrow$ INT8 or FP16

robust layers $\rightarrow$ INT4

This improves the accuracy-efficiency tradeoff.

19. Hessian View of Quantization

Quantization perturbs trained weights:

$$\hat{W} = W + \Delta W.$$

Using Taylor expansion:

$$\mathcal{L}(\hat{W}) \approx \mathcal{L}(W) + \nabla \mathcal{L}(W)^T (\hat{W} - W) + \frac{1}{2} (\hat{W} - W)^T H (\hat{W} - W).$$

Near a trained optimum, the gradient term is small.

So:

$$\Delta \mathcal{L} \approx \frac{1}{2} (\hat{W} - W)^T H (\hat{W} - W).$$

Meaning:

same-size errors can have different loss impact.

Errors in high-curvature directions are more harmful.

20. Why MSE is Not Enough

Ordinary MSE treats all quantization errors equally.

Hessian-based thinking says:

error direction matters.

A small error in a sensitive direction can hurt more than a larger error in a flat direction.

This motivates second-order PTQ methods such as GPTQ.

21. Quantization-Aware Training

Quantization-aware training (QAT) trains the model while simulating quantization.

Objective:

$$\min_W \mathcal{L}(f(x; Q(W))).$$

Main idea:

forward pass uses fake quantization

backward pass uses approximate gradients.

The model learns weights that are robust to quantization noise.

22. Fake Quantization

Fake quantization simulates quantization during training.

The values are quantized and dequantized in the forward pass:

$$x \rightarrow q \rightarrow \hat{x}.$$

But computation may still be stored in floating point during training.

Purpose:

make the model experience quantization effects before deployment.

23. Straight-Through Estimator

Rounding is not differentiable:

$$q = \text{round}\left(\frac{x}{s}\right).$$

The derivative is zero almost everywhere.

The straight-through estimator pretends the quantizer is identity during backpropagation:

$$\frac{\partial Q(x)}{\partial x} \approx 1.$$

So gradients can still flow.

24. PACT

PACT learns a clipping threshold α .

For ReLU-style activations:

$$y = \min(\max(x, 0), \alpha).$$

This is like a bounded ReLU.

If α is too small:

useful activations are clipped.

If α is too large:

quantization grid becomes too coarse.

PACT learns α to balance this tradeoff.

25. LSQ

LSQ learns the quantization step size s .

Instead of choosing s only from calibration, LSQ optimizes it during training.

Difference:

PACT learns clipping threshold

LSQ learns step size.

26. Post-Training Quantization

Post-training quantization (PTQ) quantizes a trained model with little or no retraining.

Simplest PTQ:

$$\hat{W} = s \cdot \text{round}\left(\frac{W}{s}\right).$$

This is fast but can fail at low bit-widths.

27. PTQ Improvements

Advanced PTQ improves:

- scale selection,
- rounding decisions,
- curvature awareness,
- activation outlier handling,
- cross-layer compensation.

28. AdaRound

AdaRound improves rounding decisions.

Instead of always using nearest rounding, it learns whether each value should round up or down.

Main idea:

rounding should minimize model error, not only scalar error.

29. GPTQ

GPTQ is a second-order PTQ method.

It uses Hessian information to quantize weights while minimizing expected loss increase.

Main idea:

not all weight errors are equally harmful.

GPTQ is curvature-aware.

30. SmoothQuant

SmoothQuant handles activation outliers in transformers.

It rescales activations and weights so activation quantization becomes easier.

Main idea:

move quantization difficulty from activations to weights.

Weights are easier to process offline, so this can improve transformer quantization.

31. AWQ

AWQ means Activation-aware Weight Quantization.

It uses activation behavior to identify important weights or channels.

Main idea:

protect weights that matter most for observed activations.

32. OmniQuant

OmniQuant jointly optimizes several PTQ ingredients, such as:

- scales,
- rounding,
- clipping-related parameters.

It can be viewed as a unified PTQ framework.

33. I-BERT

I-BERT focuses on integer-only Transformer inference.

It is not only about quantizing weights.

It tries to make more of the Transformer computation graph executable using integer arithmetic.

34. QLoRA

QLoRA combines quantization with LoRA fine-tuning.

Main structure:

quantized frozen base model + trainable adapters.

The base model is often stored in 4-bit precision.

The LoRA adapters are trained in higher precision.

QLoRA is a quantized fine-tuning method, not just ordinary inference quantization.

35. Quantization for Generative Models

Generative models are harder to quantize than ordinary classifiers because they are used repeatedly.

Examples:

- LLMs generate token by token,
- diffusion models denoise over many timesteps,
- flow matching models follow a numerical trajectory.

Therefore:

quantization error can accumulate over generation steps.

36. LLM Quantization

In LLMs, quantization can affect:

- attention projections,
- MLP projections,
- activations,
- embeddings,
- output heads,
- KV cache.

Challenges:

- activation outliers,
- residual pathways,
- normalization layers,
- long-context KV cache.

37. Diffusion Model Quantization

Diffusion models repeatedly apply the denoising network.

Difficulty:

activation statistics change across timesteps.

A fixed quantization scale may not work for all denoising states.

This motivates:

- timestep-aware calibration,
- mixed precision,
- protecting sensitive blocks.

38. Flow Matching Quantization

Flow matching learns a time-dependent velocity field.

Quantization can distort:

$$v_{\theta}(x, t).$$

This may change the ODE trajectory followed during generation.

Flow matching quantization is still less mature than LLM and diffusion quantization.

39. VLM Quantization

For CLIP-like dual encoders:

quantization must preserve image-text embedding alignment.

For fusion VLMs:

visual tokens, text tokens, projectors, and fusion blocks may have different sensitivities.

So VLM quantization may need modality-aware or token-aware treatment.

40. What Can Be Quantized?

Different model components can be quantized:

- weights,
- activations,
- embeddings,
- attention matrices,
- KV cache,
- projection layers,
- selected layers only.

Weights are usually easiest because they are fixed after training. Activations are harder because they depend on the input.

41. Main Quantization Tradeoff

Quantization improves efficiency but increases approximation risk.

lower precision $\Rightarrow$ better efficiency

lower precision $\Rightarrow$ higher numerical error.

The goal is to reduce precision while preserving model behavior.

42. Most Important Design Choices

Important quantization decisions:

- bit-width,
- scale,
- zero-point,
- clipping threshold,
- quantization granularity,
- calibration data,
- which layers stay high precision.

43. Most Important Equations

Affine quantization:

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right) + z, q_{\min}, q_{\max} \right)$$

Reconstruction:

$$\hat{x} = s(q - z)$$

Symmetric quantization:

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right), -Q, Q \right)$$

Signed integer range:

$$Q = 2^{b-1} - 1$$

Common symmetric scale:

$$s = \frac{\max |x|}{Q}$$

Quantization error:

$$e = x - \hat{x}$$

MSE:

$$\text{MSE} = \mathbb{E}[(x - \hat{x})^2]$$

Per-channel quantization:

$$\hat{W}_{i,:} = s_i \cdot \text{round} \left(\frac{W_{i,:}}{s_i} \right)$$

Hessian loss approximation:

$$\Delta \mathcal{L} \approx \frac{1}{2}(\hat{W} - W)^T H(\hat{W} - W)$$

QAT objective:

$$\min_W \mathcal{L}(f(x; Q(W)))$$

STE:

$$\frac{\partial Q(x)}{\partial x} \approx 1$$

PACT:

$$y = \min(\max(x, 0), \alpha)$$

LoRA/QLoRA update:

$$W' = W + BA$$

Chapter 10 Sample Questions and Answers

Question 1

What is quantization, and why is it useful?

Sample Answer:

Quantization replaces high-precision values with lower-bit approximations. It is useful because it reduces memory footprint, memory bandwidth, latency, and energy consumption. This makes models easier to deploy on limited hardware.

Question 2

What do the scale and zero-point represent in affine quantization?

Sample Answer:

The scale controls the spacing between quantization levels. The zero-point shifts the integer grid so that real zero can be represented accurately. Together, they define how real values are mapped to integer codes and reconstructed.

Question 3

Explain the tradeoff controlled by the scale s .

Sample Answer:

A small scale gives a fine quantization grid, reducing rounding error, but it also narrows the representable range and increases clipping. A large scale covers a wider range, reducing clipping, but makes the grid coarse and increases rounding error.

Question 4

Why can clipping improve quantization?

Sample Answer:

Clipping removes extreme outliers from the quantization range. This allows the scale to focus on the values that occur most often, improving resolution for typical values. However, clipping too aggressively removes useful information.

Question 5

What is the difference between per-tensor, per-channel, and group-wise quantization?

Sample Answer:

Per-tensor quantization uses one scale for the whole tensor. Per-channel quantization uses a separate scale for each channel, improving accuracy when

channels have different ranges. Group-wise quantization assigns one scale per group of channels, balancing accuracy and metadata cost.

Question 6

Why is MSE not always enough for quantization?

Sample Answer:

MSE measures the size of quantization error but ignores how harmful that error is to the loss. The Hessian view shows that errors in high-curvature directions can greatly increase loss, while errors in flat directions may be harmless. Therefore, error direction matters.

Question 7

What is the role of the Hessian in second-order quantization methods?

Sample Answer:

The Hessian measures local curvature of the loss. Second-order quantization methods use it to estimate how much a weight perturbation will increase the loss. This helps choose quantization decisions that minimize task loss rather than only parameter error.

Question 8

What is quantization-aware training?

Sample Answer:

Quantization-aware training simulates quantization during the forward pass while training. The model experiences quantization noise and learns parameters that remain effective under low-precision constraints. This usually improves low-bit deployment accuracy.

Question 9

Why is the straight-through estimator needed?

Sample Answer:

Rounding is not differentiable, so normal backpropagation cannot pass useful gradients through the quantizer. The straight-through estimator approximates the quantizer derivative as one, allowing gradients to flow while still using quantization in the forward pass.

Question 10

What is the difference between QAT and PTQ?

Sample Answer:

QAT trains or fine-tunes the model while simulating quantization effects. PTQ quantizes an already trained model with little or no retraining. QAT is usually more accurate but more expensive, while PTQ is faster and easier to apply.

Question 11

What is GPTQ’s main idea?

Sample Answer:

GPTQ is a second-order post-training quantization method. It uses Hessian information to quantize weights while minimizing expected loss increase. Its key idea is that not all weight errors are equally harmful.

Question 12

How does SmoothQuant handle transformer activation outliers?

Sample Answer:

SmoothQuant rescales activations and weights so that activation quantization becomes easier. It shifts some difficulty from activations to weights, because weights are fixed and can be processed offline.

Question 13

How is QLoRA different from ordinary quantization?

Sample Answer:

QLoRA is a quantized fine-tuning method. It keeps the base model frozen and quantized, often in 4-bit precision, while training LoRA adapter parameters in higher precision. Ordinary inference quantization usually focuses only on compressing a trained model for deployment.

Question 14

Why is quantizing generative models harder than quantizing classifiers?

Sample Answer:

Generative models are used repeatedly during generation. LLMs generate many tokens, diffusion models denoise over many timesteps, and flow matching models follow numerical trajectories. Quantization errors can accumulate across these repeated steps.

Question 15

Why is VLM quantization challenging?

Sample Answer:

VLMs contain different modalities and components. CLIP-like models must preserve image-text embedding alignment, while fusion VLMs include visual tokens, text tokens, projectors, and cross-modal fusion blocks. These components may have different quantization sensitivities.